

烟幕干扰弹的投放策略的分析与优化

摘要

本文进行投放烟幕干扰弹过程的研究,针对过程中导弹、无人机、烟幕干扰弹运动的研究,建立了描述各实体物理状态的运动学模型、有效遮蔽判定模型,解决了给定参数下有效遮蔽时长的计算问题;基于运动学模型构建了不同条件下最大化有效遮蔽时长的投放策略优化模型,解决了变量维度不断增加的策略参数寻优问题。

对于问题一,基于运动学原理,分析导弹、无人机、烟幕干扰弹的运动规律,建立**各实体运动学模型**。定义有效遮蔽为完全遮蔽,利用阴影描述建立**有效遮蔽判定模型**。对烟幕遮挡导弹视线、包裹导弹、导弹穿过烟幕三种情况分析,对目标轮廓均匀取点并设定时间步长。采用分层策略求解,通过**全局粗搜与局部二分精搜**相结合的方式,计算出给定策略下的有效遮蔽时长为 **1.3916 s**。对采样点数量进行**模型灵敏度检验**,当此数量超过 25 个后,总有效遮蔽时长稳定在一个精确数值上,证明了模型的合理与稳健性。

对于问题二,利用问题一所构建的运动学模型与有效遮蔽判定模型,以无人机航向、速度以及干扰弹的投放、起爆时间为决策变量,确定目标函数为最大化有效遮蔽时长,建立单无人机投放单枚干扰弹的**四维优化模型**。利用模拟退火法进行全局寻优求解,得到最优策略下的总有效遮蔽时长为 **4.5826 s**,无人机飞行速度为 **121.1927 m/s**。

对于问题三,利用问题一所构建的运动学模型与有效遮蔽判定模型,以无人机航向、速度以及 3 枚干扰弹的投放、起爆时间为决策变量,确定目标函数为最大化有效遮蔽时长,建立单无人机投放多枚干扰弹的**八维优化模型**。同时考虑单弹遮蔽与多弹协同遮蔽的情况下,利用遗传算法进行全局寻优求解,得到最优策略下的总有效遮蔽时长为 **7.5188 s**,其中第 1 枚烟幕弹的有效遮蔽时间区间为 **[4.8624, 7.3686]s**。针对烟幕半径对模型进行灵敏度分析,发现总有效遮蔽时长与烟幕半径呈现平滑的正相关性,验证了模型的有效性与可靠性。

对于问题四,利用问题一所构建的运动学模型与有效遮蔽判定模型,以三架无人机各自的飞行与投放参数构成的 12 个参量为变量,确定目标函数为最大化有效遮蔽时长,建立多无人机投放单枚干扰弹的**十二维优化模型**。利用**混合自适应量子遗传算法**进行全局寻优求解,得到最优策略下的总有效遮蔽时长为 **11.6673s**,无人机 FY1 的飞行速度为 **122.4957m/s**。针对导弹速度对模型进行灵敏度分析,结果中总有效遮蔽时长变化明显,证明了模型对导弹速度的高敏感性。

对于问题五,利用问题三的优化模型,将多导弹多无人机投放多弹问题简化为无人机**单独优化模型**,决策变量包括无人机航向、速度以及 3 枚干扰弹的投放、起爆时间,以最大化有效遮蔽时长为目标函数,利用**遗传算法**进行全局寻优求解,计算得到对三个导弹的**总有效遮蔽时长为 31.5572 秒**。进行无人机初始位置的鲁棒性检验,加入随机扰动后模型性能下降了 **41.60%**,体现模型对此参量较敏感,能维持一定的鲁棒性。

关键词: 阴影遮蔽判定; 模拟退火法; 改进遗传算法; 分层搜索; 高维度优化模型

一、问题重述

1.1 问题背景

随着现代战争形态的演变，攻防对抗技术飞速发展，灵活的反制手段成为防御体系建设的关键。在此背景下，具有成本低、效费比高优势的烟幕干扰技术，通过形成云团遮蔽来保护目标，已成为一种重要防御策略。

然而，烟幕干扰的过程需要预测来袭导弹的飞行轨迹、无人机的投放策略、烟幕干扰弹投放后的运动轨迹以及烟幕云团形成后的移动。传统的简单经验决策，难以应对瞬息万变的战场环境。因此，精确建模并优化烟幕干扰弹的投放策略，以寻求最优的无人机投放策略，成为提升防御系统可靠性的关键。问题目标在于利用机理建模，将运动过程抽象为数学模型，通过数据分析与算法优化，为动态干扰任务提供科学的决策方案，实现对目标的有效遮蔽时间最大化。

1.2 问题信息

- 导弹的飞行方向直指假目标，且飞行速度为 300 m/s 。
- 无人机均以 $70 \sim 140\text{ m/s}$ 的速度等高度匀速直线飞行。每架无人机的航向、速度不要求相同，且确定后不再调整。
- 每架无人机投放两枚烟幕干扰弹至少间隔 1 s 。
- 烟幕干扰弹起爆后瞬时形成球状烟幕云团，以 3 m/s 的速度匀速下沉。
- 云团有效遮蔽范围为在起爆 20 s 内，距离云团中心 10 m 的球形区域。

1.3 待求解问题

问题一: 在确定派出无人机 FY1 的情况下，根据题目给定的无人机航向、速度以及干扰弹的投放时间与起爆时间，计算投放单枚烟幕弹对导弹 M1 的有效遮蔽时长。

问题二: 基于问题一的情况，以遮蔽时间最大化为优化目标，重新设定无人机 FY1 的航向、飞行速度，烟幕干扰弹投放时间、起爆时间，对投放策略进行优化。

问题三: 在单架无人机 FY1 投放 3 枚烟幕干扰弹的条件下，对导弹 M1 实施协同干扰，设计一套完整的最优投放策略，以实现总有效遮蔽时间最大化。

问题四: 在利用无人机 FY1、FY2、FY3 各投放 1 枚烟幕干扰弹的条件下，对导弹 M1 实施协同干扰，设计 3 架无人机的最优投放策略，以实现总有效遮蔽时间最大化。

问题五: 利用 5 架无人机，在每架无人机至多投放 3 枚烟幕干扰弹的约束下，对导弹 M1、M2、M3 进行干扰，设计最优投放策略，以实现总有效遮蔽时间最大化。

二、问题分析

本套题目内包含了五个问题，围绕烟幕干扰过程中的策略参数设计展开，包括给定投放策略下、有效遮蔽时长的计算问题，再到改变策略参数设定、最大化遮挡时长的优化问题。同时增加变量，问题维度不断升级，从给定参数下求解、改变参数求解到重新设定参数寻优——层层递进，由浅入深地完成了一系列对烟幕干扰过程的研究。

2.1 问题一的分析

问题一属于确定系统下的**系统运动学仿真与几何判定问题**。其核心任务是在无人机飞行速度、方向和烟幕弹投放、起爆时间均已确定的条件下，建立一个精确描述系统中各实体(导弹、无人机、烟幕弹、烟幕云团)时空位置的数学模型，并定义有效遮蔽的几何准则^[1]，计算出总遮蔽时长。针对此类具有确定性参数和明确物理过程的求解问题，主要存在两种思路：解析求解法和时域离散仿真法。

解析求解法^[1]是通过建立一系列代数方程和不等式，直接求出遮蔽开始时刻与结束时刻的精确解析解。需要联立多实体运动方程与描述遮蔽的几何约束。但其中涉及球体烟幕-导弹-真目标构成的视线锥之间的相交关系^[1]，在时间上的表达式是极复杂的高次不等式。直接求解以获得遮蔽时间区间几乎是不可能的，模型异常复杂且难以实施。

因此，选用**时域离散仿真法**^[2]，采用时间步进的思想将总时间离散化为一系列微小的时间步长。并且对研究的关键轮廓进行均匀采样取点，作为离散样本。根据运动学方程构建**描述各实体物理状态的数学模型**，在每一离散时间点上计算出各自的位置坐标。同时利用阴影描述，构建**有效遮蔽判定模型**^{[2][3]}，判断某时刻下有效遮蔽条件是否成立。由于存在时间与轮廓的两种离散样本，结合**分层搜索与二分法**^{[2][4]}，首先进行全局粗搜，基于关键轮廓采样点离散化地确定不同时刻下的遮蔽情况，确定大致有效遮蔽时间区间；进一步进行精搜，对已确定的时间区间进行二分，判定中点的遮蔽情况，以更新有效遮蔽时间区间的上下限，最终确定较高精度下的总有效遮蔽时间区间及时长。进行**针对采样点数量的模型灵敏度检验**，通过保持其他参数不变，仅调整采样点数量，分析其对最终结果的影响，为后续的采样点数量选取提供更合理的依据。

2.2 问题二的分析

问题二是基于问题一，将确定的仿真问题转化为**多维非线性全局优化问题**^[4]。核心任务是重新设定投放策略参数，将无人机的飞行速度、飞行方向以及烟幕弹的投放和起爆时机视为待优化的**4个决策变量**，在一个受约束的多维空间内进行搜索，寻找使有效遮蔽总时长最大化的最优策略组合。针对此类以仿真模型为目标函数的**黑盒优化问题**，通常的求解方法有^[4]：网格搜索法、梯度优化算法以及启发式全局优化算法。

网格搜索法是通过将各决策变量的连续取值范围离散化，遍历所有离散点组合，调

用问题一模型计算每个组合下的有效遮蔽时长，筛选出最优解。但涉及四个连续变量，计算成本呈指数级增长^[4]，在计算中不可行。而梯度优化算法^[4]是通过计算目标函数在当前点的梯度方向来迭代更新参数，从而快速收敛至局部最优解。但问题二的目标函数没有显式解析表达式，梯度计算困难且极易陷入局部最优，无法保证找到全局最优解。

因此，选用**模拟退火算法**^{[1][4]}。通过引入随机扰动机制来跳出局部最优陷阱，具有强大的全局搜索能力。利用问题一构建好的描述各实体物理状态的数学模型与有效遮蔽判定模型，进行模拟退火迭代寻优，最终得到最大化总遮蔽时长的最优投放策略。

2.3 问题三的分析

问题三是在问题二的基础上，从单弹优化升级为**多弹协同优化**。核心是为单架无人机 FY1 设计三枚烟幕弹的投放策略，将无人机的飞行速度、方向、3 枚干扰弹的投放与起爆时机这 **8 个决策变量** 视为一个组合寻优，在一个维度更高、约束更复杂的空间内进行搜索，寻找总有效遮蔽时长最大化的最优参数组合。针对此类复杂**黑盒优化问题**，常用解法包括：网格搜索法、模拟退火等启发式算法以及遗传算法等智能算法^[5]。

其中，网格搜索法在面对问题二中四个变量已无法进行可行计算，且问题三涉及八个连续变量，使计算成本变得完全无法承受。模拟退火算法^[1]在问题二的四维优化中表现良好，但当优化维度增加到八维时，局部最优点数量和形态变得更复杂，在问题三的高维空间中可能被困于较大的局部最优区域内，全局搜索效率会显著下降。

因此，选用**遗传算法**^[4]。通过对一个包含多种策略的种群进行并行搜索，利用核心交叉算子将不同优秀策略的优良特性进行组合，创造可能更强大的新子代策略，对于处理飞行参数和多弹时序之间的高度耦合关系至关重要。利用遗传算法，将每一个八维决策向量定义为一个“个体”，将总有效遮蔽时长作为其“适应度”，不断迭代优化整个策略种群，最终寻找到最大化总有效遮蔽时长的最优投放策略。

2.4 问题四的分析

问题四是在问题一的基础上，将单架无人机干扰扩展为三架无人机协同干扰的高维优化问题。核心目标是在三架无人机分别投放一枚烟幕干扰弹的情况下，设计最优投放策略。将每架无人机的飞行速度、飞行方向、投放时间以及起爆时间共 **12 个参数** 视为待优化的决策变量，寻找使三团烟幕共同产生的总有效遮蔽时长达到最大值的策略组合。问题四的变量规模提升到了十二维，问题二中使用的模拟退火法、问题四使用的遗传算法均已经不足以处理此更高维度下的策略参数组合寻优。

因此，选用启发式全局优化算法中的**混合自适应量子遗传算法**^[6]。通过引入量子比特编码、量子旋转门更新和灾变机制，相比传统遗传算法具有更强的种群多样性和全局搜索能力，能有效避免陷入早熟收敛。

基于问题一中构建的描述各实体物理状态的数学模型与有效遮蔽判定模型，将 12 个决策变量统一编码为一个量子比特染色体，将计算出的有效遮蔽时长作为算法的适应度函数，用于评估每组策略的优劣。通过量子旋转门的迭代更新和自适应调整，引导种群向更优方向进化，最终得到最大化总有效遮蔽时长的最优投放策略。

2.5 问题五的分析

问题五是最终的规模最大的优化问题。在利用五架无人机协同应对三枚来袭导弹的情况下，要求设计一套最优的烟幕干扰弹投放策略，以实现对其目标的总遮蔽时长。其核心挑战在于决策空间的急剧膨胀和不同无人机策略之间的关联性。若将五架无人机，每架最多三枚干扰弹的所有决策变量，包括每架无人机的飞行速度、方向，以及每枚干扰弹的投放与引信时间，共 $5 \times 8 = 40$ 个变量，构建成一个单一的、庞大的优化模型。高达 40 维的搜索空间极其巨大，常规优化算法难以在有限时间内进行有效探索，极易陷入局部最优，甚至难以收敛。同时多枚烟云的遮蔽效果存在叠加情况，使目标函数具有非线性与不连续性。

为此，直接进行联合全局优化在计算上并不可行。采用一种特殊的“分层降维”的策略，将全局的高维协同优化问题，解耦为五个并行的、低维的单机最优化子问题，与问题三相似，极大地降低了求解的复杂度。通过强调在三个导弹方向下的有效遮蔽定义，参考问题三所构建的单机投多弹情况下的优化模型，在每个子问题中为单架无人机设定的目标是最大化对任一导弹所能产生的总有效遮蔽时长。最终聚合五架无人机各自独立寻优得到的最优策略，来构建完整的防御方案并评估其最终的整体遮蔽效果。

三、问题假设

为简化与完善问题信息，在此做出以下假设：

假设一：重力加速度取值为 $9.8m/s^2$ ，用于后续烟幕下沉过程计算。

假设二：以雷达发现目标的时刻作为计时起点，即 $t = 0$ 。

假设三：忽略空气阻力，简化对问题核心物理过程的分析。

假设四：参考文献 [2]，除真目标外所有实体均可视为质点，极大地简化模型，且不影响宏观轨迹的正确性。

假设五：不考虑风等气象因素对烟幕云团运动的影响。题目明确指出烟幕云团匀速下沉，其水平方向没有漂移。

假设六：参考文献 [4]，无人机和导弹的运动完全遵从设定，无任何偏差。排除随机干扰、控制误差等不确定性因素，使问题成为确定性的优化问题。

四、符号说明

分类	符号	说明	单位
物理属性	$\vec{v}_{Mk}$	导弹 Mk 飞行方向的单位向量	m
	v_M	导弹速率	m/s
	v_s	烟幕下沉速率	m/s
	r_s	烟幕半径	m
	R	真目标半径	m
	H	真目标高度	m
	g	重力加速度	m/s^2
位置坐标	P_T	真目标的位置	m
	P_F	假目标的位置	m
	P_{Mk}	导弹 Mk 的位置	m
	P_i	无人机 FY_i 的位置	m
	$P_{u(i,j)}$	无人机 FY_i 第 j 枚干扰弹的位置	m
	$P_{s(i,j)}$	无人机 FY_i 第 j 枚干扰弹的云团中心位置	m
	$P_{a(i,j)}$	无人机 FY_i 第 j 枚干扰弹的投放位置	m
	$P_{b(i,j)}$	无人机 FY_i 第 j 枚干扰弹的起爆位置	m
投放策略参数	$\vec{v}_i$	无人机 FY_i 飞行方向的单位向量	m
	θ_i	无人机 FY_i 飞行方向的角度	$^\circ$
	v_i	无人机 FY_i 的飞行速率	m/s
	$t_{a(i,j)}$	无人机 FY_i 第 j 枚干扰弹的投放时刻	s
	$t_{b(i,j)}$	无人机 FY_i 第 j 枚干扰弹的起爆时刻	s
优化目标	$T_{(i,j)}$	无人机 FY_i 第 j 个云团的有效遮挡时间	s
	T	总有效遮挡时间	s

五、问题一的模型建立与求解

问题一给出系统中的实体部件：静止真目标圆柱体、运动导弹、无人机以及其投放的烟幕干扰弹。要求在无人机 FY_1 的投放策略参数给定的情况下，计算所投放的单枚烟幕干扰弹对导弹 M_1 的有效遮蔽总时长，是一个确定性的系统仿真与几何计算问题。需要为所有实体建立运动学模型，并对有效遮蔽进行几何定义判定。通过分层搜索进行不同时刻的有效遮蔽判定，采用二分法精确遮蔽时间区间，最终计算出总有效遮蔽时长。

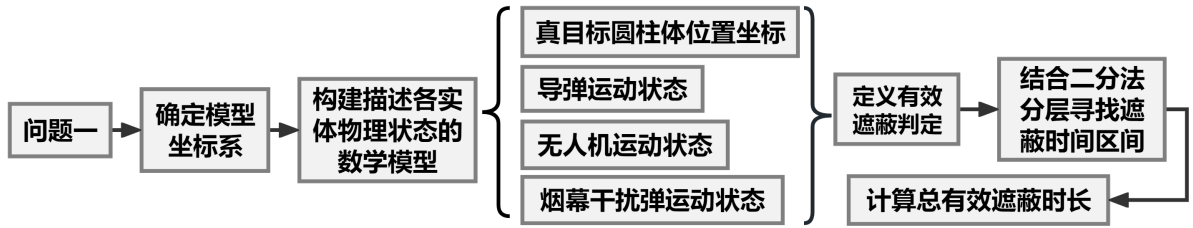


图 1 问题一框架

5.1 确定模型坐标系

根据题目描述，按照题意建立一个三维笛卡尔坐标系。如图 2 所示，以假目标中心 P_F 为坐标原点 $O(0, 0, 0)$ ，水平面为 xy 平面，垂直向上为 z 轴正方向。所有长度单位均为米，时间单位为秒。其中，真目标底面圆心 P_T 坐标为 $(0, 200, 0)$ 。

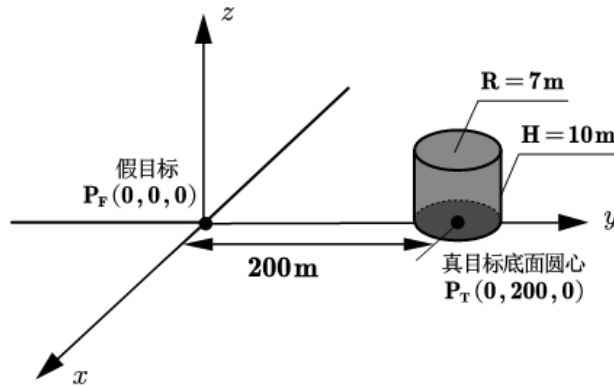


图 2 模型坐标系示意图

根据题目信息，确定无人机 FY1 的初始坐标 $P_1(0)$ 为 $(17800, 0, 1800)$ ，导弹 M1 的初始坐标 P_{M1} 为 $(20000, 0, 2000)$ ，为后续无人机与导弹的运动确定初始参数。

5.2 构建描述各实体物理状态的运动学模型

对于整个干扰过程，其中存在多实体的运动关系，需要考虑导弹、无人机、烟幕干扰弹等的运动，其中包括方向、速度大小等参量。为了更清晰的进行描述各实体不同时刻下的运动状态，采用向量进行数学化表示各自的物理状态。

5.2.1 导弹的物理运动描述

在确定的模型坐标系中，导弹的初始位置坐标向量为 $\vec{P}_{M1}(0) = (20000, 0, 2000)$ 。由于导弹飞行方向直指假目标，即指向坐标原点，则速度方向与位置坐标向量方向相反。

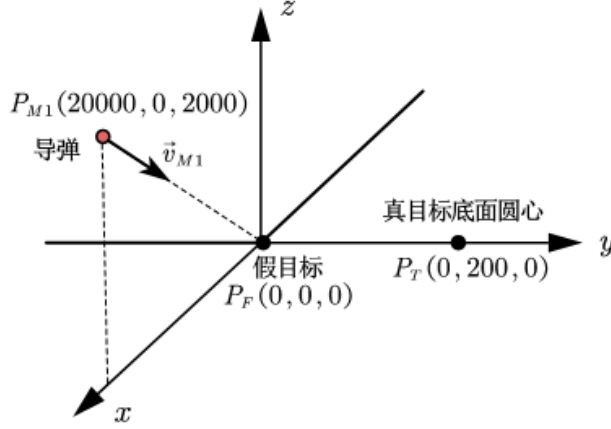


图3 导弹飞行示意图

根据图3的导弹飞行示意图分析，得到表示导弹M1速度方向的单位向量为：

$$\vec{v}_{M1} = -\frac{\vec{P}_{M1}(0)}{|\vec{P}_{M1}(0)|} \quad (1)$$

由于导弹速率恒定，得到任意时刻 t 下的导弹位置为：

$$\vec{P}_{M1}(t) = \vec{P}_{M1}(0) + v_M \cdot \vec{v}_{M1} \cdot t \quad (2)$$

将公式(1)代入公式(2)，代入已知的导弹初始位置向量 $\vec{P}_{M1}(0)$ ，以及导弹飞行速度 v_M ，得到一个以时间 t 为单一变量的导弹位置的表达式：

$$\vec{P}_{M1}(t) = \vec{P}_{M1}(0) + v_M \cdot \left(-\frac{\vec{P}_{M1}(0)}{|\vec{P}_{M1}(0)|}\right) \cdot t \quad (3)$$

5.2.2 无人机的物理运动描述

在确定的模型坐标系中，无人机FY1的初始坐标向量为 $\vec{P}_1(0) = (17800, 0, 1800)$ ，且飞行方向指向假目标，即原点。因此，得到无人机FY1飞行方向的单位向量表示为：

$$\vec{v}_1 = -\frac{\vec{P}_1(0)}{|\vec{P}_1(0)|} \quad (4)$$

由于无人机沿直线匀速运动，得到任意时刻 t 下的无人机FY1的位置为：

$$\vec{P}_1(t) = \vec{P}_1(0) + v_1 \cdot \vec{v}_1 \cdot t \quad (5)$$

在已知FY1无人机的初始位置坐标向量和飞行速率的情况下，公式(5)中唯一的变量仅为时刻 t ，即可确定任意时刻 t 下的无人机FY1的位置向量。

5.2.3 烟幕干扰弹中心的物理运动描述

烟幕干扰弹的运动分为两个时间阶段，包括烟幕干扰弹从无人机投放后的平抛运动阶段，以及起爆后形成烟幕云团的匀速下沉阶段。

1. 投放后-起爆前的平抛运动过程

根据题目信息，设定投放时刻为 t_a ，起爆时刻为 t_b ，烟幕干扰弹投放点的位置即为此刻下无人机 FY1 的位置，代入公式 (5) 得到投放点 P_a 为：

$$\vec{P}_a = \vec{P}_1(t_a) - \vec{P}_1(0) + v_1 \cdot \vec{v}_1 \cdot t_a \quad (6)$$

在此过程中，如图 4 所示，烟幕干扰弹做平抛运动，即水平方向的速度保持不变；根据假设一和四，其向下做自由落体运动，即水平与竖直方向的速度为： $\vec{v}_x = \vec{v}_1$ ， $\vec{v}_y = \vec{g}t$ 。

因此，在此过程中烟幕干扰弹的位置向量 $\vec{P}_u$ 可以表示为：

$$\vec{P}_u(t) = \vec{P}_a + v_1 \cdot \vec{v}_1 \cdot (t - t_a) + \frac{1}{2} \cdot \vec{g} \cdot (t - t_a)^2, \quad t_a \leq t \leq t_b \quad (7)$$

其中， $\vec{g}$ 表示重力加速度的向量，方向竖直向下，且速度大小为 $9.8m/s$ 。

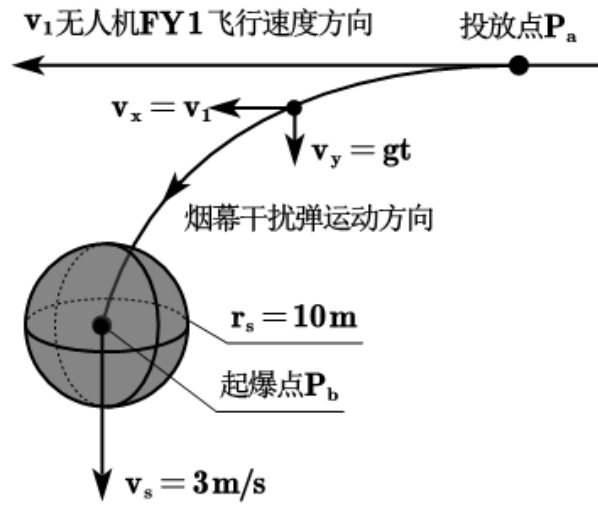


图 4 烟幕干扰弹投放后与起爆后运动示意图

2. 起爆后的匀速下沉过程

由图 4 分析，起爆时间为 t_b ，此时起爆点为上一平抛运动过程的末尾结束点，即可根据公式 (7) 代入起爆时间，得到起爆点的位置向量为：

$$\vec{P}_b = \vec{P}_a + v_1 \cdot \vec{v}_1 \cdot (t_b - t_a) + \frac{1}{2} \cdot \vec{g} \cdot (t_b - t_a)^2 \quad (8)$$

根据问题一题目中设定的投放后间隔 $3.6s$ 起爆，即 $(t_b - t_a) = 3.6s$ ，可以计算出起爆点的具体位置坐标。

起爆后，烟幕干扰弹瞬时形成一个瞬时形成烟幕云团，其有效遮蔽区域为以云团中心为球心，半径为 10 m 的球状云团。且根据题目信息，该烟幕云团以 3 m/s 的速度匀速下沉，即此时的云团的速度方向单位向量为 $\vec{v}_s = (0, 0, -1)$ ，速率大小为 $v_s = 3\text{ m/s}$ 。

因此，得到起爆后烟幕干扰弹形成的云团的位置向量 $\vec{P}_s$ 表示为：

$$\vec{P}_s(t) = \vec{P}_b + v_s \cdot \vec{v}_s \cdot (t - t_b), \quad t \geq t_b \quad (9)$$

综上所述，问题一的核心是求解在时间 t 满足有效遮蔽条件的总时长。其描述关键实体 (导弹、无人机、烟幕云团) 运动状态的核心运动方程组如下：

$$\begin{cases} \vec{P}_{M1}(t) = \vec{P}_{M1}(0) + v_M \cdot \left(-\frac{\vec{P}_{M1}(0)}{|\vec{P}_{M1}(0)|}\right) \cdot t \\ \vec{P}_1(t) = \vec{P}_1(0) + v_1 \cdot \vec{v}_1 \cdot t \\ \vec{P}_s(t) = \vec{P}_b + v_s \cdot \vec{v}_s \cdot (t - t_b), \quad t \geq t_b \end{cases} \quad (10)$$

其中，中间变量 $\vec{v}_1$ 、 $\vec{P}_a$ 、 $\vec{P}_b$ 计算的推导过程为公式 (4)(6)(8)。

5.3 构建有效遮蔽判定模型

根据实际场景分析，结合题意定义对于真目标的有效遮蔽为**完全遮蔽**，即在导弹视线中，真目标圆柱体上的任意一点均被烟幕云团遮挡。为了更清晰的解释在任意时刻下真目标受到的遮蔽情况，需要对有效遮蔽进行几何定义与简化。

5.3.1 基于阴影描述进行简化模型

真目标是一个半径 $R = 7\text{ m}$ 、高 $H = 10\text{ m}$ 的圆柱体；烟幕干扰弹起爆后形成的云团为一个半径 $r_s = 10\text{ m}$ 的球型。根据物理光学原理，可以简略模拟出烟幕云团能够遮挡掩护住真目标的圆柱体时的情况示意图，得到图 5 所示。其中，球状云团根据导弹飞行方向在地面上阴影为一个椭圆，真目标圆柱体的阴影为一个类似扇形的图形。

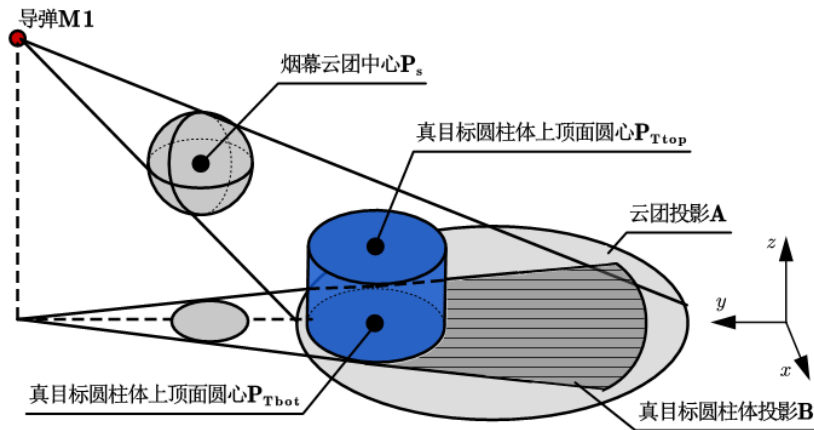


图 5 云团遮蔽情况示意图

经过分析, 可知有效遮蔽的几何意义即为真目标圆柱体的阴影均被包在云团投影内, 即可任务此时烟幕云团对真目标起到了有效遮蔽作用, 即下图 6 中的图 (a) 所示。分析其临界情况, 如下图 6 中的图 (b) 所示, 发现真目标圆柱体阴影即将超出烟幕云团阴影时, 发生接触的点不可能出现在真目标圆柱体投影的类扇形的左右两侧直线段上, 均会发生在其上下两条弧线上。

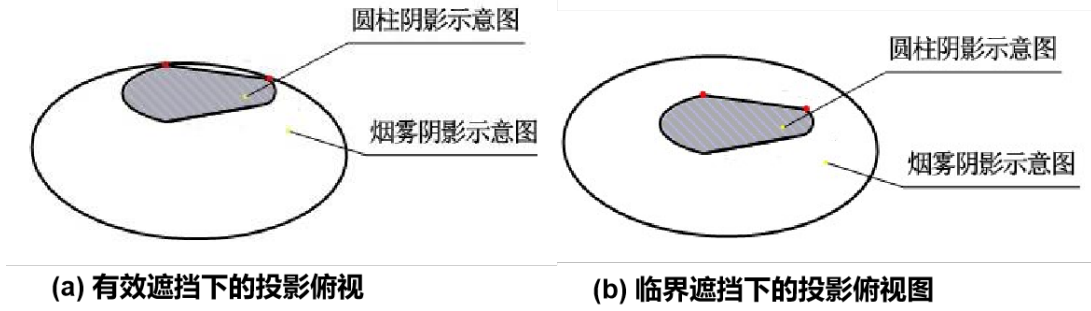


图 6 不同情况下的投影俯视图

由此, 将上下两条弧线上是否存在某点超出云团投影作为判定是否形成有效遮蔽的关键参考量。进一步分析, 可知这上下两条圆弧分别由真目标圆柱体的上顶面与下底面的投影而形成的。因此, 为了简化后续的几何遮蔽判定, 将圆柱体目标的关键轮廓, 其上下两个底面圆周进行参数化表示。

根据题目信息, 真目标圆柱体下底面圆心位于 $P_T = (0, 200, 0)$ 。因此, 得到真目标圆柱体上顶面与下底面的圆心位置向量分别为:

上顶面圆心: $\vec{P}_{Ttop} = (0, 200, 10)$, 下底面圆心: $\vec{P}_{Tbot} = (0, 200, 0)$

得到上顶面圆周上任意一点 $\vec{P}_{top}(\theta) = \vec{P}_{Ttop} + (R \cos \theta, R \sin \theta, 0)$, $\theta \in [0, 2\pi)$

得到下底圆周上任意一点 $\vec{P}_{bot}(\theta) = \vec{P}_{Tbot} + (R \cos \theta, R \sin \theta, 0)$, $\theta \in [0, 2\pi)$

5.3.2 确定判定遮蔽条件

根据实际物理场景分析, 对于整个运动过程中的烟幕干扰弹的有效遮蔽情况, 需要对导弹-云团位置的三种不同情况进行分析。

5.3.2.1 第一种情况: 烟幕云团阻挡导弹-真目标之间视线

在此情况下, 烟幕云团在导弹前的飞行路径上进行干扰, 且与导弹无接触, 即如图 7 所示。在时刻 t , 若从导弹位置指向真目标圆柱体内任意一点的视线, 均被云团球体所遮挡时, 判定该时刻为有效遮蔽。为便于计算, 将此判定条件转化为一个几何问题: 判断由点 $\vec{P}_{M1}(t)$ 和真目标轮廓 (上下圆周) 构成的视线向量, 是否与烟幕球体相交。

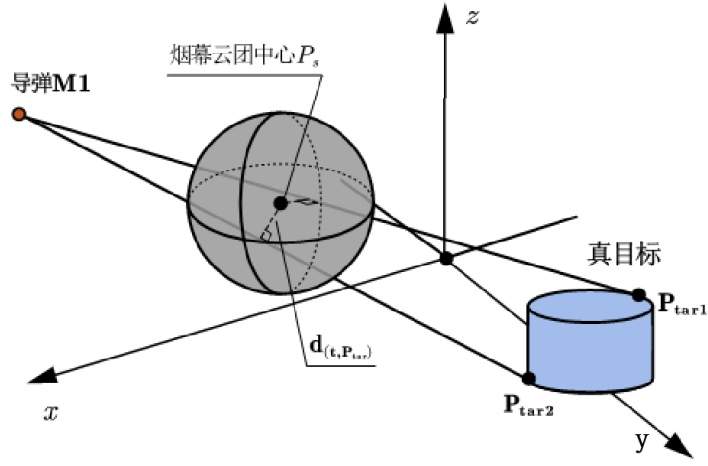


图 7 遮挡判断示意图

设定目标轮廓上任意一点的位置坐标为 $\vec{P}_{tar}$ 。在 t 时刻下，定义导弹视线为导弹 M1 与目标轮廓点的连线，向量表示为 $\vec{P}_{tar} - \vec{P}_{M1}$ ；定义导弹视线与烟幕中心的最短距离为 $d_{(t, \vec{P}_{tar})}$ ，计算公式为：

$$d_{(t, \vec{P}_{tar})} = \frac{|[\vec{P}_s(t) - \vec{P}_{M1}(t)] \times [\vec{P}_{tar}(t) - \vec{P}_{M1}(t)]|}{|\vec{P}_{tar}(t) - \vec{P}_{M1}(t)|} \quad (11)$$

则判断由点 $\vec{P}_{M1}(t)$ 和真目标轮廓构成的视线向量是否与烟幕球体相交，可以利用视线向量与烟幕中心的最短距离 $d_{(t, \vec{P}_{tar})}$ 判断。要实现烟幕云团对真目标的圆柱体实现有效遮蔽，则此距离的最大值必须小于等于该球状云团的半径 $r_s = 10m$ ，即：

$$d_{max}(t) \leq r_s = 10m \quad (12)$$

其中，满足 $d_{(t, \vec{P}_{tar})}$ 最大的目标轮廓点为 $\vec{P}_{tar}^*$ 。

同时需要注意，当烟幕云团位于导弹之后，其阴影也可以覆盖真目标圆柱体的阴影，但实际上对导弹并无干扰作用，需要对此加以约束。通过分析导弹与烟幕中心连线 $[\vec{P}_s(t) - \vec{P}_{M1}(t)]$ 、导弹与目标轮廓点连线 $[\vec{P}_{tar}^*(t) - \vec{P}_{M1}(t)]$ 两条线之间的夹角作为约束依据。其中，各个关键向量的方向与之间的夹角关系的简易示意如图 8 所示。

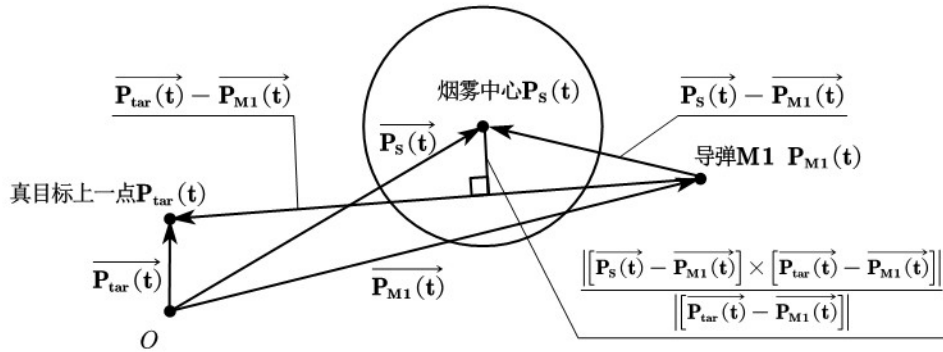


图 8 关键向量示意图

在图 8 中, $\vec{P}_{tar}(t)$ 表示 t 时刻下取的目标轮廓点位置, $\vec{P}_s(t)$ 表示 t 时刻下的烟幕云团中心点位置, $\vec{P}_{M1}(t)$ 表示 t 时刻下的导弹位置。各种带有箭头指向的线段即为不同点间的连线向量, 用于分析计算实体的位置。

根据图 8 分析可知, 当烟幕云团位于导弹前时, 导弹与烟幕中心连线、导弹与目标轮廓点连线之间的夹角必为锐角。利用夹角成锐角的两个向量的点乘值小于 0, 得到关于烟幕云团-导弹的前后位置约束条件为:

$$[\vec{P}_s(t) - \vec{P}_{M1}(t)] \cdot [\vec{P}_{tar}^*(t) - \vec{P}_{M1}(t)] > 0 \quad (13)$$

此式子表示导弹与烟幕中心连线、导弹与目标轮廓点连线两条线夹角为锐角, 即烟幕云团位于导弹位置之前。

5.3.2.2 第二种情况: 烟幕云团将导弹包裹住

在此情况下, 导弹位于烟幕云团内部, 即导弹的所有视线均被完全遮挡, 因此必定视线有效遮蔽。此时可以转化为一个几何问题: 判断导弹质点位置是否位于云团内。

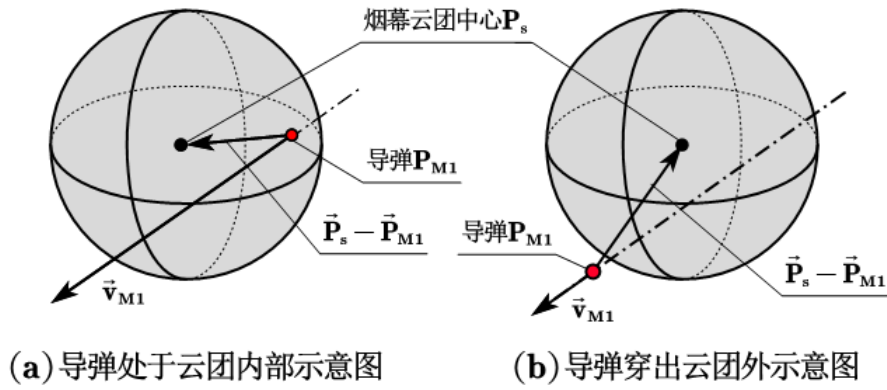


图 9 导弹-云团不同情况下的位置示意图

根据图 9 中 (a) 分析, 可以计算导弹与烟幕云团中心的距离 $|\vec{P}_s(t) - \vec{P}_{M1}(t)|$, 通过与烟幕云团半径 $r_s = 10m$ 进行比较: 若距离小于此半径, 则导弹此时位于烟幕云团内部, 发生了有效的完全遮挡。并且当导弹穿出云团后, 就不可能再发生有效遮蔽。

即此情况下有效遮蔽的充要条件为:

$$|\vec{P}_s(t) - \vec{P}_{M1}(t)| \leq r_s = 10m \quad (14)$$

5.3.2.3 第三种情况: 导弹穿出云团

在此情况下, 导弹穿出云团, 即烟幕云团在导弹之后, 如图 9 中 (b) 所示。当导弹在飞行方向上位于烟幕云团球体前方时, 烟幕云团无法对导弹进行干扰, 因此对于真目标无法实现有效遮蔽。即在此情况下, 判定无有效遮蔽。

5.3.2.4 判定遮蔽条件的条件汇总

综合上述三种情况，在时刻 t 下烟幕云团对真目标圆柱体实现有效遮蔽的条件为：

$$\text{云团阻挡导弹-真目标之间视线: } \begin{cases} d_{max}(t) \leq r_s = 10m \\ [\vec{P}_s(t) - \vec{P}_{M1}(t)] \cdot [\vec{P}_{tar}^*(t) - \vec{P}_{M1}(t)] > 0 \end{cases} \quad (15)$$

$$\text{或 导弹在云团内: } |\vec{P}_s(t) - \vec{P}_{M1}(t)| \leq r_s = 10m \quad (16)$$

其中， $d_{max}(t)$ 为取目标轮廓上所有点计算 $d_{(t, \vec{P}_{tar})}$ 后找到的最大值。

在时刻 t 下满足其中任一情况，即可确定此时烟幕干扰弹对导弹 M1 发生有效遮蔽。

5.4 采用分层策略进行二分法数值求解

在有效遮蔽判断模型中，不仅需要对不同时刻下的系统状态进行仿真，同时需要对圆柱体的上下底面周长上的各个点进行判断。因此具有两个维度下的计算搜索，直接求解遮蔽起始与终止时刻的解析解是不可行的。因此，采用分层策略，通过全局粗搜与局部二分精确求解相结合的方式，以较高的精度确定总有效遮蔽时长。

5.4.1 确定已知参数

在问题一题目中，明确给出了系统内的各实体的部分速度等各参数，将已知参数汇总成表格，得到如下表 1 所示。

表 1 问题一模型已知参数汇总

参数符号	参数值	参数符号	参数值
$\vec{P}_{M1}(0)$	$(20000, 0, 2000)m$	r_s	$10m$
$\vec{P}_1(0)$	$(17800, 0, 1800)m$	v_s	$3m/s$
v_M	$300 m/s$	R	$7m$
v_1	$120m/s$	H	$10m$
t_a	$1.5s$	$\vec{P}_{Tbot}$	$(0, 200, 0)m$
$t_b - t_a$	$3.6s$	g	$9.8m/s^2$

5.4.2 进行分层策略下的全局粗搜

1. 连续目标轮廓离散化

为进行数值判定，在 t 时刻下将真目标的连续轮廓离散化。初步对真目标圆柱体的上、下底面圆周分别均匀选取 $N_p = 100$ 个采样点，构成上轮廓点集 P_{top} 与下轮廓点集

P_{bot} 。总轮廓点集定义为 $P = P_{top} \cup P_{bot}$ 。即在时刻 t 下，分别对这 200 个离散点进行仿真，利用构建好的有效遮挡判断模型进行判断是否全部点均发生有效遮蔽。

2. 设定时间步长进行粗搜确定大致时间区间

基于构建好的描述各实体物理状态的数学模型，从 $t=0$ 时刻开始仿真，以 $0.5s$ 为初始步长，不断前推计算、遍历搜索，直到全局粗搜搜索到两对相邻时间的遮蔽状态转变点：未有效遮蔽时间点 t_1 、有效遮蔽时间点 t_2 、有效遮蔽时间点 t_3 、未有效遮蔽时间点 t_4 。随后在这四个时间点构成的两个区间 $[t_1, t_2]$ 、 $[t_3, t_4]$ 内进行二分搜索。

5.4.3 二分法精搜精确时间区间

根据全局粗搜结果，得到有效遮蔽时间区间上下限的大致取值范围。如图 10 所示，需要在 $[t_1, t_2]$ 、 $[t_3, t_4]$ 内进行精细搜索，以一定的精度确定有效遮蔽时间区间的上下限。



图 10 二分搜索示意图

对于寻找总有效遮蔽时间区间下限，在 $[t_1, t_2]$ 内进行。取其区间的中点 $t_{mid} = \frac{t_1+t_2}{2}$ ，判断 t_{mid} 时刻下的烟幕遮蔽情况。若对真目标实现有效遮蔽，则更新下一次搜索区间的上限为此 t_{mid} ；若未实现完全遮蔽，则更新下一次搜索区间的下限为此 t_{mid} 。

对于寻找总有效遮蔽时间区间上限，在 $[t_3, t_4]$ 内进行。与上一步骤相同，取其区间中点 $t_{mid} = \frac{t_3+t_4}{2}$ ，判断 t_{mid} 时刻下的遮蔽情况。若对真目标实现有效遮蔽，则更新下一次搜索区间的下限为 t_{mid} ；若未实现完全遮蔽，则更新下一次搜索区间的上限为 t_{mid} 。

分别对上述两个步骤进行 **25 次循环迭代**，最终得到在两个区间内分别取到一个最终值，计算得到的总有效遮蔽时间区间为 $[t_{start}, t_{end}]$ ，总遮蔽时长为：

$$T = t_{start} - t_{end} \quad (17)$$

5.5 总有效遮蔽时长展示与分析

构建好各实体物理状态模型和有效遮蔽判定模型，采用分层数值求解策略，对问题一中的有效遮蔽时长进行精确计算。通过仿真计算，得到有效遮蔽开始时刻与终止时刻的高精度数值解： $t_{start} \approx 8.0564s$ 、 $t_{end} \approx 9.4480s$ ，即**总有效遮蔽时长为 1.3916 s**。

结果表明，烟幕弹于 $t_b = 5.1s$ 起爆后，并非立即产生有效遮蔽。经过一段时间的下沉运动，遮蔽窗口在 $t \approx 8.0564$ 秒时才开启，说明烟幕云团的运动轨迹与导弹视线在该

时刻首次形成几何遮蔽条件。而窗口在约 1.39 秒后关闭，原因可能是导弹已飞过最优点位，或烟幕云团相对于动态变化的视线已偏移。

5.6 针对采样点数量进行模型灵敏度检验

为便于进行视线遮蔽的几何判断，模型中引入一个数值计算参数：**均匀轮廓采样点数**。为检验模型的稳健性，并验证求解中采用 100 个点参数选择的合理性，进行针对采样点数量的检验，探究最终结果对轮廓采样点数量变化的敏感程度。

保持问题一中的所有参数不变，将目标轮廓的单圈均匀采样点数 N 作为唯一变量，使其在整数区间 $[1, 100]$ 内遍历。对于每一个 N 值，完整运行一次分层数值求解，记录相应的总有效遮蔽时长 $T(N)$ 。将得到的数据点绘制成折线图，得到图 9 所示。

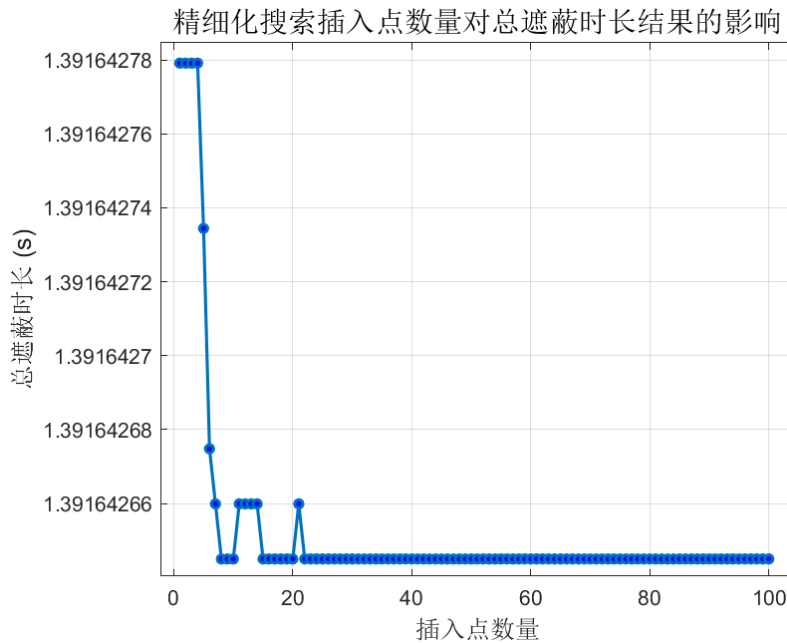


图 11 精细搜索插入点数量-总遮蔽时长结果的关系图

理论上，采样点数目的选择会直接影响计算的精度与效率：点数过少可能导致对目标几何特征的描述不足，从而产生计算误差；点数过多则会无谓增加计算负担。

观察图 11 可知，当采样点数非常少时，结果表现出明显的不稳定性。例如，在 N 从 5 增加到 15 的过程中，结果在 1.39164265s 到 1.39164278s 之间跳变，是因为过少的离散点无法精确地代表连续的圆周轮廓，导致在某些临界时刻遮蔽区间的端点计算产生影响，造成结果的抖动。而当采样点数 N 超过大约 25 个之后，总遮蔽时长的结果迅速收敛，并稳定在一个精确数值上。之后即使继续将点数增加到 100，结果再无任何明显变化。表明当采样点足够密集时，离散点集已能够以高保真度来近似模拟连续的物理轮廓。因此，在后续有效遮蔽判定可以适当地减少采样点数量，取 50 个采样点即已足够。

5.7 问题一小结

针对问题一的确定系统下有效遮蔽时长求解，构建了一个描述导弹、真目标与烟幕云团空间关系的动态运动学模型，和基于阴影描述的有效遮蔽判定模型。采用全局粗搜索-局部二分法的分层策略，通过仿真计算精确地确定了在给定条件下，有效遮蔽的起始与结束时刻，从而得出总有效遮蔽时长为 1.3916 秒。为验证该结果的可靠性，对模型中关键的标轮廓离散化点数进行了检验。分析表明，尽管在采样点数极少时计算结果存在预期内的波动，但当采样点数超过 25 个之后，总遮蔽时长的计算结果便迅速收敛并稳定在一个精确值上，充分证明了所选参数的合理性以及模型自身的数值稳定性。

六、问题二的模型建立与求解

问题二相当于问题一的优化问题，在问题一的基础上将无人机的飞行方向、速度以及烟幕弹的投放和起爆时机等关键参数视为待优化的决策变量。目标是寻找一组最优参数组合，以实现对导弹 M1 的有效遮蔽总时长的最大化。将沿用问题一中建立的各实体运动学模型和有效遮蔽判定模型作为基础，并确定最大化有效遮蔽时长的目标函数。考虑多个连续变量，且解空间复杂，选择采用模拟退火法的全局优化方法，在约束范围内对策略参数进行迭代搜索，最终确定最大化有效遮蔽时长的投放策略。

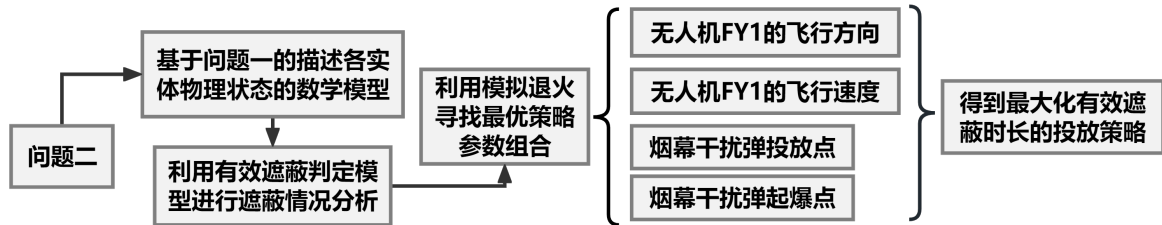


图 12 问题二框架

6.1 基于问题一模型确定核心运动方程组与遮蔽判定

根据问题一中构建好的各实体的物理运动状态模型，其描述关键实体(导弹、烟幕云团)运动状态的核心运动方程组如下：

$$\begin{cases} \vec{P}_{M1}(t) = \vec{P}_{M1}(0) + v_M \cdot \left(-\frac{\vec{P}_{M1}(0)}{|\vec{P}_{M1}(0)|}\right) \cdot t \\ \vec{P}_1(t) = \vec{P}_1(0) + v_1 \cdot \vec{v}_1 \cdot t \\ \vec{P}_s(t) = \vec{P}_b + v_s \cdot \vec{v}_s \cdot (t - t_b), \quad t \geq t_b \end{cases} \quad (18)$$

其中，中间变量的推导过程与问题一不同，需要输入的参量包括无人机 FY1 的航向 $\vec{v}_1$ 、速度 v_1 以及烟幕弹的投放时刻 t_a 、起爆时机 t_b ，即为优化模型的决策变量。在此

运动方程基础上，利用问题一的有效遮蔽判定模型，对真目标圆柱体的上下面圆周进行均匀取采样点，同时设定时间步进步长。结合判定遮蔽的条件，确定有效遮蔽时间点。

6.2 基于模拟退火法建立单弹投放策略的优化模型

6.2.1 确定决策变量向量

在问题二中，决策变量为无人机 FY1 的飞行方向 $\vec{v}_1$ 、速度 v_1 以及烟幕弹的投放时刻 t_a 、起爆时机 t_b 。因此，将无人机 FY1 的策略参数定义为四元决策向量 $\mathbf{x}$ ：

$$\mathbf{x} = (v_1, \theta_1, t_a, \Delta t) \quad (19)$$

其中， v_1 为无人机 FY1 的飞行速率； θ_1 为无人机 FY1 的飞行水平方向角，以 x 轴正方向为 0，逆时针为正， $\theta_1 \in [0, 2\pi)$ ； t_a 为烟幕弹的投放时刻； Δt 为烟幕弹从投放到起爆的间隔时间，即 $\Delta t = t_b - t_a$ 。

6.2.2 确定优化目标函数与约束条件

优化目标是最大化有效遮蔽总时长 T 。该时长是关于决策变量 $\mathbf{x}$ 的函数。沿用问题一中建立的各实体运动学模型和有效遮蔽判定模型，定义一个函数 $f(\mathbf{x})$ ，根据方程组 (10) 与中间变量推导公式 (4)(6)(8) 计算在给定策略 $\mathbf{x}$ 下的有效遮蔽总时长。因此，目标函数可以表示为：

$$\max_{\mathbf{x}} T = \max_{\mathbf{x}} f(v_1, \theta_1, t_a, \Delta t) \quad (20)$$

其中，无人机 FY1 的航向不再固定指向假目标。其方向向量 $\vec{v}_1$ 由方向角 θ_1 确定：

$$\vec{v}_1 = (\cos \theta_1, \sin \theta_1, 0) \quad (21)$$

将上式 (21) 代入公式 (5)，无人机在任意时刻 t 的位置 $\vec{P}_1(t)$ 更新为：

$$\vec{P}_1(t) = \vec{P}_1(0) + v_1 \cdot \vec{v}_1 \cdot t = \vec{P}_1(0) + (v_1 \cos \theta_1, v_1 \sin \theta_1, 0) \cdot t \quad (22)$$

此式子 (22) 代回方程组 (18)，作为后续计算烟幕弹投放点 $\vec{P}_a$ 和起爆点 $\vec{P}_b$ 的基础。

对于方位角进行系统情况分析，发现方位角的大小，决定无人机与导弹的飞行方向为同向或反向，即无人机朝向真目标方向或朝与导弹对碰方向前进，在坐标系下表示为无人机朝 X 正半轴过负半轴飞行。由此，需要对这两种不同情况进行讨论。

6.2.2.1 无人机朝 X 正半轴飞行的情况方向

在此情况下，方位角大小约束为 $0 < \theta_1 \leq \frac{\pi}{2} \cup \frac{3\pi}{2} \leq \theta < 2\pi$ 。考虑题目约束包括对无人机飞行速率的约束以及对于时间的非负约束。结合上述决策变量与目标函数，得到无人机朝 X 正半轴飞行的情况下，优化模型核心为：

$$\max_{\mathbf{x}} T = \max_{\mathbf{x}} f(v_1, \theta_1, t_a, \Delta t) \quad (23)$$

$$\text{决策变量: } \mathbf{x} = (v_1, \theta_1, t_a, \Delta t) \quad (24)$$

$$s.t. \begin{cases} 70 \leq v_1 \leq 140, & (0 < \theta_1 \leq \frac{\pi}{2}) \cup (\frac{3\pi}{2} \leq \theta < 2\pi) \\ 0 < t_a, & t_a < t_b < \sqrt{\frac{2z_1}{g}}, \quad 0 < \Delta t \end{cases} \quad (25)$$

6.2.2.2 无人机朝 X 负半轴飞行的情况方向

在此情况下，方位角大小约束为 $\frac{\pi}{2} < \theta_1 < \frac{3\pi}{2}$ 。考虑题目约束包括对无人机飞行速率的约束以及对于时间的非负约束。结合上述决策变量与目标函数，得到无人机朝 X 负半轴飞行的情况下，优化模型核心为：

$$\max_{\mathbf{x}} T = \max_{\mathbf{x}} f(v_1, \theta_1, t_a, \Delta t) \quad (26)$$

$$\text{决策变量: } \mathbf{x} = (v_1, \theta_1, t_a, \Delta t) \quad (27)$$

$$s.t. \begin{cases} 70 \leq v_1 \leq 140, & \frac{\pi}{2} < \theta_1 < \frac{3\pi}{2} \\ 0 < t_a, & t_a < t_b < \sqrt{\frac{2z_1}{g}}, \quad 0 < \Delta t \end{cases} \quad (28)$$

其中， $70 \leq v_1 \leq 140$ 为无人机飞行速度约束； $t_a < t_b < \sqrt{\frac{2z_1}{g}}$ 物理意义为烟幕干扰弹必须在投放后才发生起爆，并且需要在落地之前发生起爆。 z_1 为无人机 FY1 的位置坐标的沿 z 轴方向的高度，即其距离地面的距离。

6.3 利用模拟退火法全局寻优求解

问题二可能存在多个局部最优解，且目标函数 $f(\mathbf{x})$ 没有显式的解析表达式，其值需要通过完整的系统仿真计算得出。由此选用模拟退火算法进行全局寻优，引入随机因素，能够以一定概率跳出局部最优，从而有更大机会找到全局最优解。

Step1: 初始化

随机生成一组初始策略参数 $\mathbf{x}_0$ 作为当前解，并设置初始温度 $T'_{start} = 1000$ 、终止温度 $T'_{end} = 10^{-4}$ 和温度衰减系数 $\alpha = 0.99$ 。计算当前解对应的目标函数值 $f(\mathbf{x}')$ 。

Step2: 迭代搜索

在当前温度 T' 下，对当前解的四个分量分别施加服从正态分布的随机扰动，生成一个邻域内的新解，并确保新解满足所有约束条件。计算新解的目标函数值 $f(\mathbf{x}_{new})$ 与目标值的增量 ΔT 。利用有效遮蔽判定模型，以 **0.5s** 作为时间步进步长。对真目标圆柱体的上下面圆周分别进行均匀取 **50 个采样点**，并将终止迭代次数设置为 **15 次**。

若 $\Delta T > 0$ ，说明新解更优，则无条件接受新解，令 $\mathbf{x}_{current} = \mathbf{x}_{new}$ 。

若 $\Delta T \leq 0$ ，说明新解较差，则以概率 $P = \exp(\frac{\Delta T}{T})$ 接受新解。

Step3: 降温：完成当前温度下的足够多次迭代后，进行降温： $T' = \alpha \cdot T'$ 。

Step4: 终止：重复 Step2 和 3，直到温度降低至 T'_{end} 。算法终止，输出整个搜索过程中记录下的最优策略参数 $\mathbf{x}_{best}$ 及其对应的最大有效遮蔽时长 T_{max} 。

6.4 最优投放策略展示与分析

通过上述优化模型和模拟退火算法求解，得到实现最长有效遮蔽时长的最优策略参数组合；包括无人机朝 X 正半轴和负半轴两种不同情况下的最优投放策略，具体参数分别如表 2、3 所示。

表 2 无人机朝 X 正半轴情况下的最优投放策略参数

参数 / 结果	数值
最大有效遮蔽时长	4.5826 s
无人机飞行速度	121.1927 m/s
无人机飞行方向	5.0600°
烟幕弹投放时间	0.9936 s
烟幕弹起爆时间	0.9946 s
最佳投放点	(17919.9500, 10.6300, 1800.0000) m
最佳起爆点	(17919.9500, 10.6300, 1800.0000) m

表 3 无人机朝 X 负半轴情况下的最优投放策略参数

参数 / 结果	数值
最大有效遮蔽时长	4.4513 s
无人机飞行速度	102.3730 m/s
无人机飞行方向	178.6400°
烟幕弹投放时间	0.0001 s
烟幕弹起爆时间	2.9557 s
最佳投放点	(17799.9000, 0.0000, 1800.0000) m
最佳起爆点	(17497.4100, 7.1700, 1757.2000) m

表 2 和表 3 分别展示了两种情况下的最优解，即无人机速度 X 轴分量为负和正两种情况，分别得到两个局部最优解。比较最大有效遮蔽时长，发现无人机速度 X 轴分量为正的策略更优。因此，最终选取表 2 展示策略作为全局最优策略。

对表 2 中的最优策略参数进行分析，其为真目标提供 $4.5826s$ 的有效遮蔽，相较于问题一中的 $1.3916s$ ，提升了约 230%，显著增强了干扰效果。

其中，无人机选择 121.19 m/s 的较高飞行速度，接近速度上限，并在受领任务后不到 $1s$ 即投放烟幕弹。这表明最优策略是以最快速度抵达一个稍偏离初始位置但对遮蔽极为有利的投放点。且航向并非直指假目标，而略微向 Y 轴正方向偏转 5.0600° ，使无人机能够将烟幕弹精准投放到导弹视线与真目标之间，从而形成最有效的几何遮蔽。

6.5 问题二小结

针对问题二中最大化有效遮蔽时长的优化问题，在问题一的模型基础上，构建了以无人机投放策略参数为决策变量的非线性优化模型。为求解，引入了模拟退火全局优化算法，对包含飞行速度、方向、投放与起爆时机的多维参数空间进行寻优。最终确定了一组最优投放策略，其中单枚烟幕弹的最大有效遮蔽时长可达 4.5826 秒。对最优策略分析，其核心为利用无人机快速抵达关键区域并即刻生成烟幕，以实现干扰效能的最大化。验证了所建优化模型的有效性，为制定高效的烟幕干扰策略提供了量化依据。

七、问题三的模型建立与求解

问题三相当于问题二的升级问题，从单弹干扰提升至多弹协同干扰。加入三枚烟幕弹投放时间与起爆时间的协同参数，维度更高、耦合性更强。目标是寻找一组最优的协同策略组合，以实现导弹 M1 的有效遮蔽总时长的最大化。沿用问题一中建立的各实体运动学模型和有效遮蔽判定模型，以最大化总有效遮蔽时长为目标函数。考虑连续变量增多至八个，解空间复杂度呈指数级增长，采用遗传算法，在约束范围内对协同策略参数进行迭代搜索，最终确定最大化有效遮蔽时长的多弹协同投放策略。

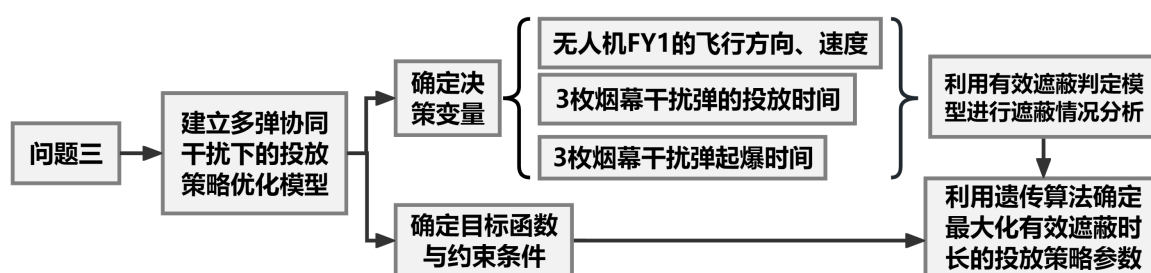


图 13 问题三框架

7.1 基于遗传算法建立多弹协同干扰下的投放策略优化模型

7.1.1 确定决策变量向量

问题三增加无人机 FY1 所投放的烟幕干扰弹数量，因此在多弹投放策略下其中的参数数量增加，包括基础的无人机飞行方向、速度，以及 3 枚烟幕干扰弹分别的投放时间与起爆时间。设定最优的 8 维决策向量为 $\mathbf{x}$ ，包括无人机的飞行与投放策略。

$$\mathbf{x} = [v_1, \delta_\theta, t_{a_1}, t_{b_1}, \Delta t_{12}, t_{b_2}, \Delta t_{23}, t_{b_3}] \quad (29)$$

其中， v_1 为无人机飞行速度，取值范围为 $[70, 140](m/s)$ 。 δ_θ 为无人机飞行方向相对基准方向（无人机初始位置指向假目标）的偏差角，取值范围为 $[-\frac{\pi}{4}, \frac{\pi}{4}](rad)$ ，且最终航向角为 $\theta = \theta_1 + \delta_\theta$ 。 t_{a_1} 为第一枚弹的投放时刻。 $t_{b_1}, t_{b_2}, t_{b_3}$ 分别为三枚弹各自的起爆时刻。 $\Delta t_{12}, \Delta t_{23}$ 即为第二、三枚弹相对于前一枚弹的投放时间间隔。

7.1.2 确定优化目标函数与约束条件

优化目标是最大化总有效遮蔽时长 T 。该时长是关于决策变量 $\mathbf{x}$ 的函数。沿用问题一中建立的各实体运动学模型和有效遮蔽判定模型，定义一个函数 $f(\mathbf{x})$ ，根据方程组 (10) 与中间变量推导公式 (4)(6)(8) 计算在策略 $\mathbf{x}$ 下的总有效遮蔽时长。因此目标函数可以表示为：

$$\max_{\mathbf{x}} T = \max_{\mathbf{x}} f(v_1, \delta_\theta, t_{a_1}, t_{b_1}, \Delta t_{12}, t_{b_2}, \Delta t_{23}, t_{b_3}) \quad (30)$$

且约束条件与问题二 (27)(28) 相同，此外需特别加上关于投弹间隔约束： $1 < \Delta t_{12}, 1 < \Delta t_{23}$ 。

7.1.3 计算单弹遮蔽区间与总有效遮蔽时间区间

对于每一枚烟幕弹 $j \in \{1, 2, 3\}$ ，其形成的烟幕有效遮蔽时间最大为 $20s$ ，即在有效时间区间 $[t_{b_j}, t_{b_j} + 20]$ 内，通过时间步进 $\Delta t = 0.5s$ 来判断其在每一时刻 t 是否对真目标构成有效遮蔽。根据有效遮蔽判定模型，得到第 j 枚弹产生的有效遮蔽时间集合为：

$$T_j = \{t \mid t \in [t_{b_j}, t_{b_j} + 20]\}$$

$$\text{云团在导弹前: } \begin{cases} d_{max}(t) \leq r_s = 10m \\ [\vec{P}_s(t) - \vec{P}_{M1}(t)] \cdot [\vec{P}_{tar}^*(t) - \vec{P}_{M1}(t)] > 0 \end{cases} \quad (31)$$

$$\text{或导弹在云团内: } |\vec{P}_s(t) - \vec{P}_{M1}(t)| \leq r_s = 10m \quad (32)$$

即在 t 时刻下，只要满足其中任意一种情况，即可确定此时第 j 枚烟幕干扰弹单独对真目标产生了有效遮蔽，时刻 t 为有效遮蔽时间集合内元素。

根据实际情况分析,问题三中投放的3枚烟幕干扰弹不仅存在**单一烟幕**即可完成对真目标的有效遮蔽,还存在**多个烟幕协同**对于真目标完成有效遮蔽的情况。绘制双烟幕有效遮蔽示意图,得到图14。

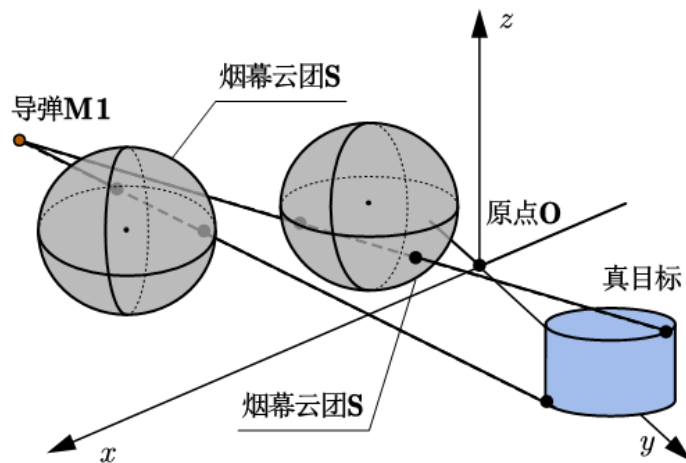


图14 双烟幕有效遮蔽示意图

如图14所示,多个烟幕协同完成有效遮蔽的情况即为其中单独一个烟幕无法完全阻挡导弹-真目标之间的视线,但可以进行部分视线的遮挡。因此,在多个烟幕协同,对其他烟幕遮挡不到的部分进行补充遮挡,也可以实现对真目标的有效遮蔽。

综合上述**单独遮蔽**与**协同遮蔽**两种情况,分别计算3枚烟幕干扰弹的单独有效遮蔽时间区间后,得到3枚烟幕干扰弹作用下的总遮蔽时间集合是各个单弹遮蔽时间集合的并集 $T = T_1 \cup T_2 \cup T_3$ 。

通过标准区间合并算法,将 T 中所有可能重叠的子区间合并,得到一组最终的、互不重叠的有效遮蔽区间。目标函数 $f(\mathbf{x})$ 即为这些合并后区间的总长度。

7.2 利用遗传算法进行全局寻优求解

在确定参数化策略框架后,由于决策变量向量包含多个连续变量,并且各决策变量之间存在复杂的非线性关系,目标函数的响应曲面可能存在大量局部最优解。遗传算法通过其群体并行搜索的特点,相比模拟退火算法能更有效地探索解空间。因而选择遗传算法进行全局寻优,避免困入局部最优。结合题目,具体求解流程如下。

Step1: 确定基因编码

将一个完整的投放策略抽象为一个“基因”。这个基因就是由之前定义的所有决策变量构成的向量 $\mathbf{x} = [v_1, \delta_\theta, t_{a_1}, t_{b_1}, \Delta t_{12}, t_{b_2}, \Delta t_{23}, t_{b_3}]$, 构成了一个完整“DNA”。给定任意一个合法的向量 $\mathbf{x}$, 就可以生成唯一的完整无人机FY1投放3枚导弹情况下的投放策略。搜索空间即由这些向量 $\mathbf{x}$ 在边界约束内构成的多维空间。

Step2: 确定适应度函数

适应度函数是驱动 GA 进化的核心，用于评估每一个“基因” $\mathbf{x}$ 的优劣程度。适应度被直接定义为该策略所能实现的总有效遮蔽时长 T 。

$$\text{Fitness}(\mathbf{x}) = f(\mathbf{x}) = T(\mathbf{x})$$

对于种群中的每一个体 $\mathbf{x}$ ，调用问题一中建立的物理仿真与遮蔽判断模型，完整计算其对应的 T 作为其适应度值。适应度值越高的个体，代表其对应的投放策略越优秀。

Step3. 确定遗传算法参数

为实现上述遗传算法，设定关键参数，如下表 4 所示。

表 4 遗传算法参数设置

参数类别	参数值	参数类别	参数值
种群规模	200	变异概率	0.5
最大迭代次数	3000	选择父代时的候选个体数量	3
每代保留的最优个体数	10	停滞阈值	50 代
交叉概率	0.8		

Step4. 优化求解

首先进行初始化，在可行域内随机生成大小为 N_p 的初始种群，计算种群中每个个体 $\mathbf{X}$ 的适应度值 $\text{Fitness}(\mathbf{X})$ 。

进入进化循环，当未达到最大进化代数 G_{max} 时，重复执行“选择父代种群 → 对父代进行交叉和变异操作，生成子代种群 → 评估子代种群的适应度 → 将父代与子代合并，通过精英保留和选择策略形成新一代种群”的操作，直至算法跳出循环。

算法终止后，输出最终种群中适应度最高的个体作为最优设计基因 $\mathbf{x}^*$ ，及其对应的最大适应度值 T^* 。使用最优设计基因 $\mathbf{x}^*$ 作为设计规则，生成最终投放策略。

7.3 最优投放策略展示与分析

基于各实体运动学模型与有效遮蔽判定模型，通过遗传算法进行迭代寻优，最终得到了一套实现最长有效遮蔽时长的最优协同投放策略，保存至文件 result1.xlsx 中。其具体参数与效果如表 5 所示。并将不同枚烟幕干扰弹的有效遮蔽区间可视化，得到图 15。

分析表 5 中的参数数据，所展示的策略可实现高达 **7.4369 秒** 的总有效遮蔽时长，相较于问题二中的单弹最优策略，其性能实现质的飞跃，展现了多弹协同干扰的高性能。

表 5 最优投放策略参数

分析对象	参数	数值
总体情况	最大有效遮蔽时长	7.5188 s
无人机 FY1	飞行速度	140.0000 m/s
	飞行方向	179.8687°
第 1 枚烟幕弹	投放点	(17799.9860, 0.0000, 1800.0000) m
	起爆点	(17799.9720, 0.0000, 1800.0000)m
	有效遮蔽时长	2.5062 s
	有效遮蔽时间区间	[4.8624, 7.3686]s
第 2 枚烟幕弹	投放点	(17458.4429, 0.7827, 1800.0000) m
	起爆点	(16763.5967, 2.3750, 1679.2966)m
	有效遮蔽时长	3.4085 s
	有效遮蔽时间区间	[7.4530, 10.8615]s
第 3 枚烟幕弹	投放点	(17099.6518, 1.6049, 1800.0000)m
	起爆点	(16278.8760, 3.4858, 1631.5809)m
	有效遮蔽时长	1.6040 s
	有效遮蔽时间区间	[10.8652, 12.4692]s

其中三枚弹单独产生的有效遮蔽时长分别为 3.3584 秒、2.8070 秒和 1.3033 秒，其算术和为 7.4687 秒。这个数值与最终协同产生的总时长 7.4369 秒极为接近，**差值仅为 0.0318 秒**。这展示了最优策略并非通过烟幕云团的大量重叠来延长单个遮蔽窗口的持续时间，而是采取一种极其高效的时间接力模式，精确控制每一枚弹的投放间隔与起爆时间，实现时序上的精准衔接，最大化地利用了每一枚弹 20 秒的有效周期，避免因过度重叠而造成的时长浪费，最终形成一个连贯的遮蔽效果。

而下图 15 更直观地展示了无人机 FY1 最优投放策略在时间维度上的具体效果。以时间为横轴，标示了整个有效遮蔽的发生序列。图中每一个橙色条块代表一枚烟幕弹从形成有效遮蔽到失效的完整时间窗口。条块的起始位置代表该弹开始产生有效遮蔽的时刻，其长度反映单枚弹的有效遮蔽持续时长，具体数值也标注在条块内部。图中蓝色条块展示了三枚弹共同作用下的有效遮蔽时间段，是所有橙色条块在时间轴上进行逻辑运算后的结果，代表了只要有任意一枚弹在起作用，导弹视线就会被遮蔽。

图中用绿色椭圆圈出了第二枚弹遮蔽效果末期与第三枚弹遮蔽效果初期的衔接部分，精确地指出这两枚弹在时间上存在一个极短的重叠期，时长仅为 0.0037 秒。在整个干扰过程中，会出现第二枚弹与第三枚弹同时满足真目标起到有效遮蔽作用的情况。

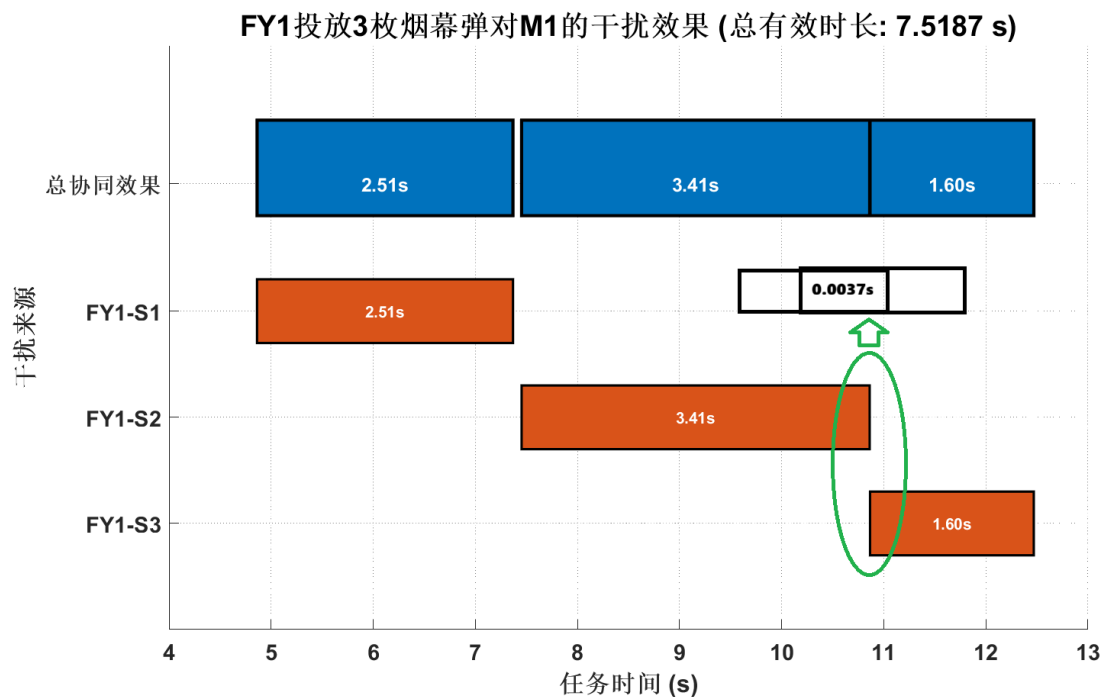


图 15 最优投放策略下的有效遮蔽区间展示

7.4 针对烟幕半径进行模型灵敏度检验

为检验模型的鲁棒性与现实合理性,对核心参数烟幕半径进行了敏感性分析。保持其他参数与条件不变,仅对烟幕半径进行上下浮动 20% 的调节,并计算相应变化的总有效遮蔽时长,绘制成关系折线图,得到图 16 所示。

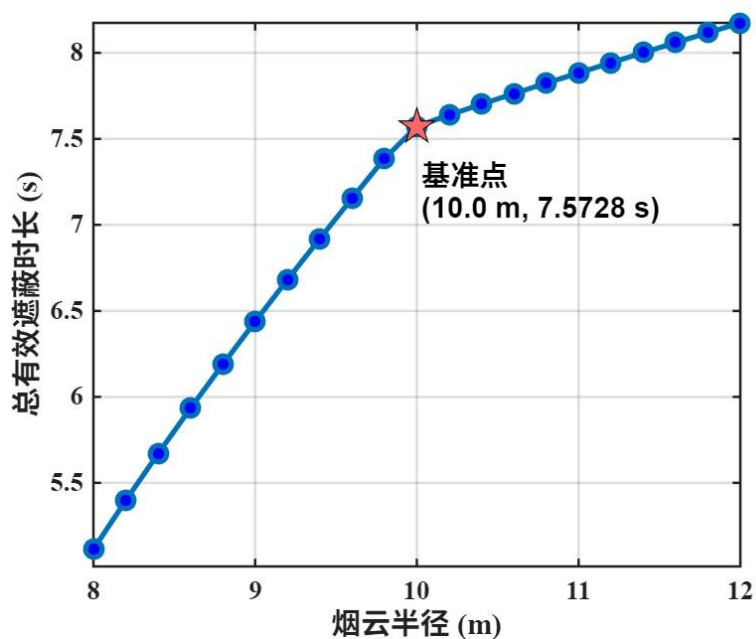


图 16 烟幕半径-总遮蔽时长关系图

分析图 15 可知，在固定最优投放策略不变的前提下，总有效遮蔽时长与烟幕半径呈现出显著的、平滑的正相关性。当半径在 $[8,12]m$ 的区间内变化时，有效时长也随之稳定地增减，其变化趋势完全符合“烟幕越大，遮蔽效果越好”的物理直觉。且曲线中未出现任何突变或不合理的振荡。此检验有力地证明，所构建的模型性能稳定，对关键参数的扰动响应合理，验证了模型的有效性与可靠性。

7.5 问题三小结

针对问题三的多弹协同干扰下的最大化有效遮蔽时长的优化问题，在问题一的模型基础上，构建了一个以无人机飞行与三弹投放时序为决策变量的优化模型。引入遗传算法进行求解，对包含无人机飞行策略和三枚弹各自投放与起爆参数的八维参数空间内寻优。最终确定了最优协同投放策略，三枚烟幕弹协同产生的最大有效遮蔽时长可达 7.4369 秒。对最优解通过对三枚弹投放时序的精准控制，实现遮蔽效果在时间维度上的无缝衔接，形成时间接力式的连续干扰，从而最大化地利用了每枚弹的有效性。

八、问题四的模型建立与求解

问题四要求利用 FY1、FY2、FY3 三架无人机，各投放 1 枚烟幕干扰弹，对 M1 导弹实施干扰，目标是使总有效遮蔽时间最长。在问题一所建立的运动学和几何遮蔽模型基础上，构建一个多无人机协同投放策略的优化模型，其中以三架无人机各自的飞行与投放参数构成的 12 个参量为变量，以最大化总有效遮蔽时长为目标函数。根据问题的变量维度量级，选择混合自适应量子遗传算法进行求解，全局寻找最优策略参数组合。基本解题框架与问题三相同，即图 13 所示。

8.1 基于混合自适应量子遗传算法建立多无人机投单弹的优化模型

8.1.1 确定决策变量向量

对于该问题，需要为三架无人机 FY1、FY2、FY3 分别确定其飞行和投放策略。因此，决策变量是一个包含 12 个元素的向量 $\mathbf{x}$ ：

$$\mathbf{x} = [v_1, \theta_1, t_{a1}, \Delta t_1, v_2, \theta_2, t_{a2}, \Delta t_2, v_3, \theta_3, t_{a3}, \Delta t_3] \quad (33)$$

其中，对于第 i 架无人机 ($i \in \{1, 2, 3\}$)， v_i 为无人机 i 的飞行速率。 θ_i 为无人机 i 的飞行方向角，定义为其速度向量在 xy 平面与 x 轴正方向的夹角。 t_{ai} 为无人机 i 投放烟幕干扰弹的时刻。 Δt_i 为第 i 枚烟幕弹的引信时间，即从投放到起爆的时间间隔。 $t_{bi} = t_{ai} + \Delta t_i$ 为第 i 枚烟幕弹的起爆时刻。

8.1.2 确定优化目标函数与约束条件

推广问题一中的运动学模型来描述整个干扰过程中导弹、三架无人机及其投放的烟幕弹，真目标的物理运动状态，即可得到描述关键实体 (导弹 M1、无人机 FY i 、烟幕云团 i) 运动状态的核心运动方程组如下：

$$\begin{cases} \vec{P}_{M1}(t) = \vec{P}_{M1}(0) + v_M \cdot \left(-\frac{\vec{P}_{M1}(0)}{|\vec{P}_{M1}(0)|}\right) \cdot t \\ \vec{P}_i(t) = \vec{P}_i(0) + v_i \cdot \vec{v}_i \cdot t \\ \vec{P}_{si}(t) = \vec{P}_{bi} + v_s \cdot \vec{v}_s \cdot (t - t_{bi}), \quad t \geq t_{bi} \end{cases} \quad (34)$$

通过上述方程组，即可确定任意 t 时刻下的系统静态几何构型。基于问题三的多枚烟幕干扰弹分析，依照图 14，对单烟幕遮蔽与多烟幕协同遮蔽两种情况进行考虑分析。进一步利用有效遮蔽判定模型中的公式 (30)(31)，进行确定该时刻下真目标是否被烟幕云团完全遮蔽。并且确定目标函数。

根据问题三中对于多枚烟幕干扰弹遮蔽时间区间取并集、去重复交集的步骤，对问题四中三架无人机分别投放的三枚干扰弹进行分析。并且整个过程的约束与问题三所述相同，分别对多个无人机进行分析即可。综上所述，问题四的优化模型可以表述为：

$$\max_{\mathbf{x}} T = \max_{\mathbf{x}} f(v_1, \theta_1, t_{a1}, \Delta t_1, v_2, \theta_2, t_{a2}, \Delta t_2, v_3, \theta_3, t_{a3}, \Delta t_3) \quad (35)$$

$$s.t. \begin{cases} 70 \leq v_i \leq 140, \quad 0 \leq \theta_i < 2\pi \\ 0 < t_{ai}, \quad t_{ai} < t_{bi} < \sqrt{\frac{2z_i}{g}}, \quad 0 < \Delta t \end{cases} \quad \forall i \in \{1, 2, 3\} \quad (36)$$

8.2 利用混合自适应量子遗传算法进行全局寻优求解

经过对问题四的分析，相较于问题三变量规模与量级增加，计算更复杂。继续使用问题三中的遗传算法虽可以获得结果，但计算效率较低。考虑量子计算在并行性和多样性方面的优点，决定采用混合自适应量子遗传算法来进行更高效地求解。

Step1: 确定量子位编码

采用量子位染色体对解空间中的决策变量进行编码。每个决策变量 $v_i, \theta_i, t_{ai}, \Delta t_i$ 等均由 L 个量子比特来表示。一个量子比特 $|\psi\rangle$ 可处于 $|0\rangle$ 和 $|1\rangle$ 的线性叠加态：

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle \quad (37)$$

其中， α 和 β 是复数，称为概率幅，满足 $|\alpha|^2 + |\beta|^2 = 1$ 。

问题四中整个 12 维的解向量 $\mathbf{x}$ 由一条长度为 $n_{genes} = 12 \times L$ 的量子染色体表示。

Step2: 确定适应度函数

优化目标是最大化总有效遮蔽时长 $T_{total}(\mathbf{x})$ ，即将适应度函数 $Fitness(\mathbf{x})$ 定义为：

$$Fitness(\mathbf{x}) = T(\mathbf{x}) \quad (38)$$

最终目标是找到使 $Fitness(\mathbf{x})$ 最大的决策向量 $\mathbf{x}$ 。对于的每一条量子染色体 $\mathbf{x}$ ，调用问题一中建立的物理仿真与遮蔽判断模型，完整计算其对应的 T 作为其适应度值。适应度值越高的个体，代表其对应的投放策略越优秀。

Step3. 核心进化算子

针对问题四的复杂组合寻优，设计以下三种算子来驱动种群进化。

(1) 基于适应度的自适应量子旋转门

量子旋转门是更新量子比特概率幅、引导种群向更优解进化的核心操作^[5]。基于自适应旋转策略，旋转门的角度 $\Delta\theta$ 根据当前个体适应度与当代种群最优适应度的差距动态调整^{[7][5]}：当个体解的质量与最优解相差较远时，采用较大的旋转角进行大步长搜索；当个体解接近最优解时，则采用较小的旋转角进行精细的局部搜索，以提高解的精度。

(2) 自适应量子交叉算子

为加速优良基因的传播，引入交叉操作直接作用于量子染色体，采用轮盘赌法选择父代，并设计了自适应交叉概率^[7]：适应度高于种群平均水平的个体将以更高的概率进行交叉，从而使优质的解片段能更快地在种群中扩散和重组。

(3) 大灾变机制

为防止算法陷入局部最优，引入大灾变机制^[8]，监测算法的收敛状态。如果最优解连续多代没有改善，则判定算法陷入了停滞。将随机重置种群中一部分个体的量子染色体^[8]，使其恢复到初始状态，旨在跳出局部最优，重新在新区域进行探索。

Step4. 确定算法关键参数

表 6 量子遗传算法参数设置

参数类别	参数值	参数类别	参数值
种群规模	500	变异概率	0.5
最大迭代次数	2000	量子位编码长度	12
基础交叉概率	0.7	大灾变停滞代数	50 代
精英交叉概率	0.9	自适应旋转门角度	[0.001,0.05]

Step5. 迭代寻优

进入进化循环，当未达到最大进化代数 G_{max} 时，重复对当前量子种群 $Q(t-1)$ 中的所有染色体进行计算每个解对应的总遮蔽时长 T 。比较当前代所有个体的适应度，并与历史全局最优解进行比较，更新并记录全局最优解及其对应的量子染色体。

在进化操作中，通过大灾变检查：判断是否满足停滞条件，若满足则触发大灾变操作。进行量子交叉与量子旋转，以设定的基础交叉概率 = 0.7 与自适应旋转门角度，对种群执行自适应量子更新。并且以 0.9 的精英交叉概率将记录的全局最优量子染色体替换当前种群中的最差个体，最终形成新一代的量子种群。

Step6. 循环结束后输出结果

循环结束后，将记录的全局最优解进行解码，得到最终投放策略。

8.3 最优投放策略展示与分析

利用前述构建的混合自适应量子遗传算法，进行模型求解。算法经过充分迭代收敛后，得到了一组最优协同策略参数，其性能指标如表 6 所示，策略内参数如表 7 所示。将此策略下的关键参数进行可视化，得到图 17 的二维俯视展示图。

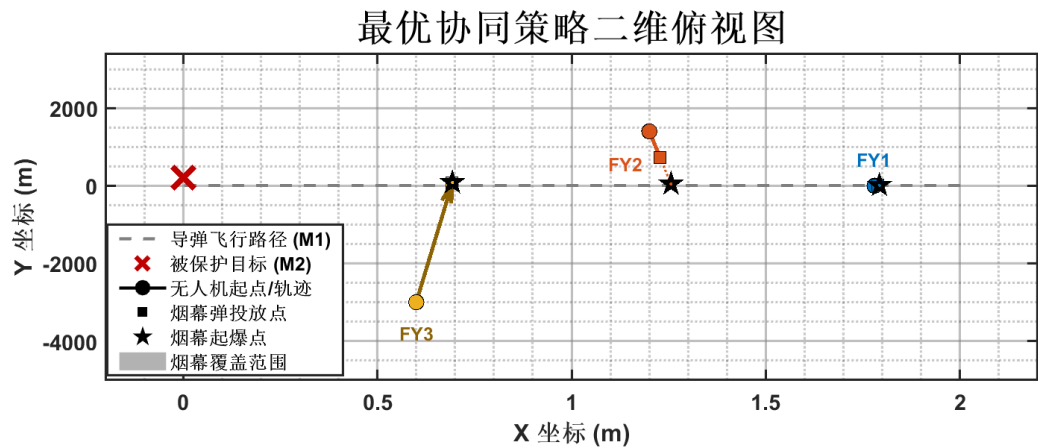


图 17 最优协同策略二维俯视示意图

观察图 17，图中的虚线表示导弹飞行路径，不同颜色的点线则表示了不同无人机的运动路径，而星形标记代表烟幕干扰弹的起爆点。图中直观地展示了最优协同策略下，三架无人机的运动轨迹与烟幕云团的空间布局。从俯视图中可以清晰地看到，所有三枚烟幕弹的起爆点均精确地分布在导弹的飞行路径上。

无人机 FY1 从远端沿导弹路径方向运动，在距离真目标最远的位置形成了有效遮蔽；而无人机 FY2 和 FY3 则分别从导弹路径的上下两侧向中间靠拢，在飞行路径的中段和末端实现有效遮蔽，最大化地利用了空间，确保在导弹飞行的不同阶段都能形成有效遮蔽，从而直观地解释了该协同策略能够取得最长总有效时长的原因。

进一步展示在该最优协同策略下的有效遮蔽时长与具体参数，如表 7、表 8 所示。

表 7 策略有效时长性能

结果	数值
最大总有效遮蔽时长	11.6673s
FY1 烟幕独立有效时长	4.5861s
FY2 烟幕独立有效时长	3.9701s
FY3 烟幕独立有效时长	3.1110s

分析表 7 结果，通过三架无人机的协同配合，实现了长达 11.6673 秒的总有效遮蔽时长。其中，三个独立时长的总和（11.6672 秒）与最大总有效时长几乎完全相等，表明此最优策略是一种时间接力式遮蔽，三个烟幕云团的有效作用时间区间重叠极小。

表 8 问题四求解的最佳协同策略参数

分析对象	参数	数值
无人机 FY1	飞行速度	122.4957m/s
	飞行方向	5.2308°
	投放点坐标	(17877.5800, 7.1000, 1800.0000)m
	起爆点坐标	(17923.8400, 11.3400, 1799.3000)m
无人机 FY2	飞行速度	139.9487m/s
	飞行方向	292.4396°
	投放点坐标	(12277.4200, 728.2500, 1400.0000)m
	起爆点坐标	(12559.4400, 45.3400, 1263.4200)m
无人机 FY3	飞行速度	123.0085m/s
	飞行方向	73.1868°
	投放点坐标	(6927.1000, 68.1400, 700.0000)m
	起爆点坐标	(6931.1500, 81.5600, 699.9400)m

表 8 中的最优协同策略参数展示了三架无人机在对导弹 M1 进行干扰时的目标。无人机 FY1 的飞行方向角极小，其主要沿初始方向快速飞行。相比之下，无人机 FY2 与 FY3 则执行了明显的机动变向。这组参数表明最优策略并非简单的集群行动，而是通过精确计算，让每架无人机到达不同的空间位置，以形成覆盖范围更广的协同遮蔽效果。

8.4 针对导弹速度进行模型灵敏度检验

为了检验所求得的最优协同策略在多变环境下的适应性和稳定性，需要对模型进行灵敏度分析。在模型中，导弹的飞行速度是一个已知常量。但在实际中是一个核心且难以精确预知的变量，直接决定了无人机进行干扰的时间窗口。因此，评估基于导弹速度为 300 m/s 常量下计算出的最优策略，在面对此导弹速度参数波动时的性能变化，对于衡量该策略的鲁棒性至关重要。

将问题四求解得到的最优协同策略内的其他 12 个参数保持不变。以基准导弹速度 300 m/s 为中心，在其上下浮动 10% 的范围内（即 270 m/s 至 330 m/s）进行系统性调整。在每个调整后的导弹速度下，重新运行仿真模型，计算并记录在该固定协同策略下所能取得的总有效遮蔽时长，最终绘制出策略性能随导弹速度变化的曲线，得到图 18。

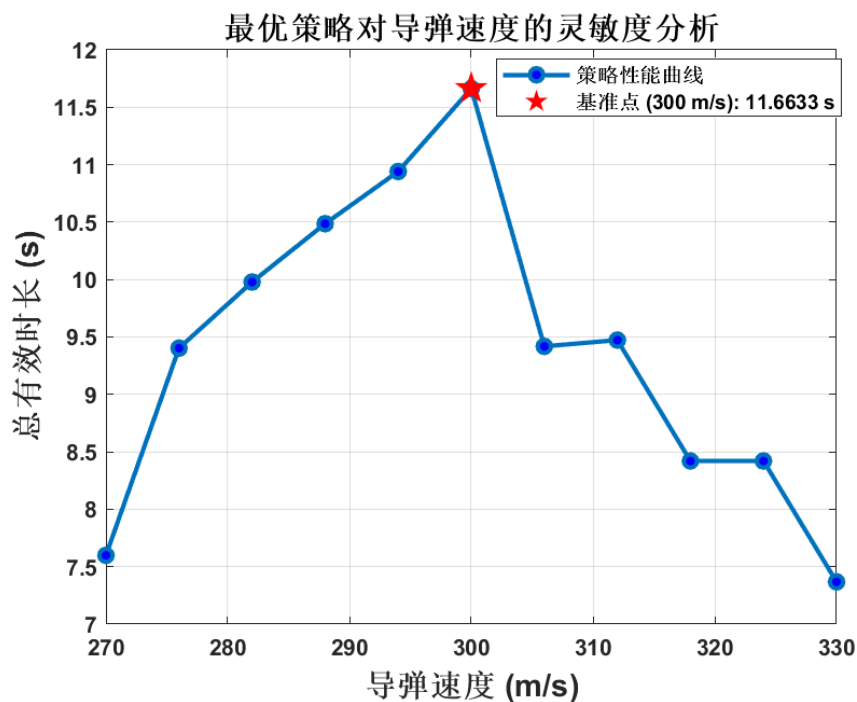


图 18 导弹速度-总有效遮蔽时长关系折线图

图中红色星点标示了基准导弹速度为 300 m/s 下，最优策略能够实现 11.6633 秒的总有效时长，是曲线的峰值点。当导弹实际速度偏离 300 m/s 时，总有效时长均出现下降。具体来看，性能曲线呈现出以 300 m/s 为中心的尖峰形态：当导弹速度降低或升高时，遮蔽时长都迅速减少。例如，当速度降至 270 m/s 或增至 330 m/s 时，总有效时长已分别锐减至约 7.6 s 和 7.4 s ，性能下降超过 35%。说明设计策略参数是经过精确计算的，用以在特定时间窗口拦截特定速度的目标。一旦目标速度变化，其到达预定拦截点的时间就会偏离，使得遮蔽效果大打折扣。这一结果充分证明了该最优策略对目标速度参数的高度敏感性。

8.5 问题四小结

针对问题四提出的三架无人机协同干扰以最大化总遮蔽时长的任务，基于问题一构建一个包含 12 个决策变量的多无人机协同投放优化模型。为求解此高维非线性模型，运用混合自适应量子遗传算法，融合了量子交叉、自适应旋转门和灾变机制，能够有效平衡全局探索与局部开发，以搜索最优协同策略。最终计算得到的最优协同策略可实现长达 11.6673 秒的总有效遮蔽，并呈现出一种时序接力式的协同模式。

九、问题五的模型建立与求解

问题五要求在利用 5 架无人机，且每架无人机至多投放 3 枚烟幕干扰弹的条件下，实施对 3 枚导弹的干扰。将五架无人机进行整体分析优化，整个模型的变量数目会增加

至少 40 个，求解维度剧烈增加，几乎不可能完成计算。根据问题三的单无人机投多弹的情况，将问题五中的多架无人机进行单独分析，即将问题整体拆分理解为五个问题三进行并行求解，确定每一架无人机的最优投放策略。最终进行汇总，得到全局下包括所有无人机的最优投放策略，计算五架无人机协同作用下对每个导弹有效遮蔽时长之和，即总有效遮挡时长。

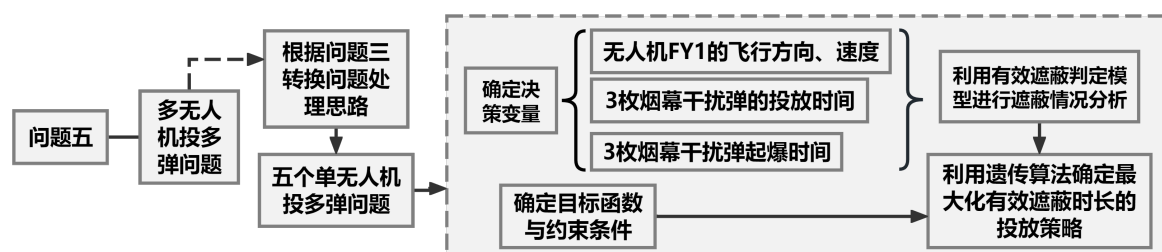


图 19 问题五框架

9.1 简化多无人机投多弹的优化模型

问题五的系统中包含五架无人机的飞行参数与 3 枚烟幕干扰弹的投放、起爆参数，变量数目会增加至 $8 \times 5 = 40$ 个，求解维度剧烈增加，几乎不可能一次性完成统一计算优化。为了进行正确且高效的求解，需要对此规模庞大的优化模型进行**简化或拆分**。

9.1.1 多个导弹方向下的有效遮蔽判定简化

问题五要求对 3 枚导弹 M1、M2、M3 进行干扰，且这 3 枚导弹的初始位置与飞行方向均不相同，即对于真目标，存在三个方向上的袭击威胁。经过初步猜想与推测，同时满足三个方向上的干扰条件过于苛刻，难以实现。因此，有效遮蔽并非定义为三个导弹方向上均可以实现有效遮蔽。而是与前几问判定相同，对单一导弹分析，将有效遮蔽定义为在某一导弹视线内，即**实现某一导弹方向上的对真目标的完全遮蔽，就判定为对此导弹有效遮蔽**。为了尽可能的保护真目标，就需要每个导弹的有效遮挡时长之和最大。

9.1.2 基于问题三下的优化模型简化

对于问题五，模型优化思路是，将复杂的总有效遮蔽时长优化问题，分层转化为独立的，简洁的每个飞机对真目标遮挡时间最大化，将核心目标总有效遮蔽时长转化单个飞机对目标遮蔽时长，通过对问题三现有模型改进，得到高效、准确的数学模型。

在利用 5 架无人机进行干扰且每架无人机至多投放 3 枚干扰弹的条件下进行分析；而问题三为对于单架无人机，至多投放 3 枚干扰弹的条件下进行分析。若对问题五进行常规求解会导致维度巨大，计算困难。针对维度爆炸的难题，选取用一种特殊的“分层

降维”的策略，将全局的高维协同优化问题，解耦为五个并行的、低维的单机最优化子问题：将问题五的优化模型拆分为**五架不同无人机下的单独优化模型**。

目标在于对每一架无人机设定单独的、不受其他无人机影响的飞行与投放策略，使每一无人机的干扰过程实现对每个导弹有效遮蔽时长之和最大化。若无人机在某时刻下对真目标实现有效遮蔽，则必定对三个导弹中的某一个实施了干扰。可以计算在 5 架无人机单独作用下三个导弹分别受到最大干扰时长，得到每个导弹被特定烟雾的时间遮蔽区间，通过对每个飞机对该导弹造成的时间遮蔽取并集运算，得到对于该导弹的有效遮蔽时长。将三个导弹的有效干扰时长相加，得到总有效干扰时长，即核心优化目标。

9.1.2.1 确定决策变量

对无人机 FY i 进行分析 ($i = 1, 2, 3, 4, 5$)，初步假设必定会投放 3 枚烟幕干扰弹。根据问题三中模型建立的过程可以确定其决策变量有 8 个，即决策向量为八维向量：

$$\mathbf{x} = [v_i, \delta_{\theta_i}, t_{a(i,1)}, t_{b(i,1)}, \Delta t_{12}, t_{b(i,2)}, \Delta t_{23}, t_{b(i,3)}] \quad (39)$$

其中， v_i 为无人机 FY i 飞行速度。 δ_{θ} 为无人机飞行方向相对基准方向（无人机初始位置指向假目标）的偏差角，且最终航向角为 $\theta_i = \theta_0 + \delta_{\theta}$ 。 $t_{a(i,1)}$ 为第一枚弹的投放时刻。 $t_{b(i,1)}, t_{b(i,2)}, t_{b(i,3)}$ 分别为三枚弹各自的起爆时刻。 $\Delta t_{12}, \Delta t_{23}$ 即为所投放的第二、三枚弹相对于前一枚弹的投放时间间隔。

9.1.2.2 确定目标函数与约束条件

优化目标是最大化对真目标遮蔽时长 T 。该时长是关于决策变量 $\mathbf{x}$ 的函数。沿用问题一中建立的各实体运动学模型和有效遮蔽判定模型，定义一个函数 $h(\mathbf{x})$ ，计算在策略 $\mathbf{x}$ 下的导弹受干扰时长。因此目标函数可以表示为：

$$\max_{\mathbf{x}} T = \max_{\mathbf{x}} h(v_i, \delta_{\theta_i}, t_{a(i,1)}, t_{b(i,1)}, \Delta t_{12}, t_{b(i,2)}, \Delta t_{23}, t_{b(i,3)}) \quad (40)$$

其中，约束条件与问题二 (27) 相同。并且在此情况下对于有效遮蔽判定条件为：对于每一枚烟幕弹 $j \in \{1, 2, 3\}$ ，其形成的烟幕有效遮蔽时间最大为 $20s$ ，即在有效时间区间 $[t_{b(i,j)}, t_{b(i,j)} + 20]$ 内，通过时间步进 $\Delta t = 0.5s$ 来判断其在每一时刻 t 是否对真目标构成有效遮蔽。根据有效遮蔽判定模型，在真目标圆柱体上下底面圆周分别取 $N = 30$ 得到无人机 FY i 所投放的第 j 枚烟幕干扰弹，针对导弹 Mk 产生的有效遮蔽时间集合为：

$$T_k(i, j) = \{t \mid t \in [t_{b(i,j)}, t_{b(i,j)} + 20]$$

$$\text{云团在导弹前: } \begin{cases} d_{max}(t) \leq r_s = 10m \\ [\vec{P}_{s_j}(t) - \vec{P}_{Mk}(t)] \cdot [\vec{P}_{tar}^*(t) - \vec{P}_{Mk}(t)] > 0 \end{cases} \quad (41)$$

$$\text{或导弹在云团内: } |\vec{P}_{s_j}(t) - \vec{P}_{Mk}(t)| \leq r_s = 10m \quad (42)$$

其中， $t_{a(i,1)}, t_{b(i,1)}$ 分别为无人机 FY i 投放的第 j 枚烟幕干扰弹的投放时间与起爆时间。 $d_{max}(t)$ 为取目标轮廓上所有点计算 $d_{(t, \vec{P}_{tar})}$ 后找到的最大值。 $\vec{P}_{s_j}$ 为第 j 枚干扰弹形成的烟幕云团中心位置。 $\vec{P}_{Mk}$ 为导弹 Mk 的位置。

即在 t 时刻下，只要任一无人机所投放的任一干扰弹，对于任一导弹满足上述任意一种情况，即可确定此时刻下实现了有效遮蔽，时刻 t 为有效遮蔽时间集合内元素。

分别计算 5 架无人机各自投放的 3 枚烟幕干扰弹的单独有效遮蔽时间区间后，得到总遮蔽时间集合是各个单独遮蔽时间集合的并集。通过标准区间合并算法，将 T 中所有可能重叠的子区间合并，得到一组最终的、互不重叠的有效遮蔽区间。目标函数 $T = f(\mathbf{x})$ 即为这些合并后区间的总长度。

其中需要特别说明，为减少决策变量，在模型中初步假设每一架无人机必定都会投放 3 枚烟幕干扰弹。但在实际中，可能存在并不需要将 3 枚干扰弹全部投放的情况，即某些干扰弹对真目标的有效遮蔽作用为 0，属于多余投放。因此，在模型计算结果中，根据每一枚导弹的有效遮蔽时长，若其值为 0，则意味着此枚导弹投放无意义，即在最终投放策略内此枚干扰弹将不计入对应无人机的投放数量，剔除无效的干扰弹。

9.2 利用遗传算法进行全局寻优求解

在利用遗传算法进行迭代求解的过程中，其步骤与 7.2 节中问题三中遗传算法的步骤相同。其中不同在于根据问题五的要求与规模，重新设定关键参数，如表 9 所示。

表 9 遗传算法参数设置

参数类别	参数值	参数类别	参数值
种群规模	200	变异概率	0.5
最大迭代次数	3000	选择父代时的候选个体数量	3
每代保留的最优个体数	10	停滞阈值	50 代
交叉概率	0.8		

输入表 9 中各关键参数，进行问题三相同的遗传算法步骤，生成最终投放策略。

9.3 最优投放策略展示与分析

通过对每架无人机执行独立的遗传算法优化，为五架无人机 FY1、FY2、FY3、FY4、FY5 分别规划了最优的飞行与投放策略，以协同应对三枚导弹 M1, M2, M3。所有无人机均根据优化结果调整了其飞行方向与速度，并在特定的时间窗口内，以精心计算的间隔，投掷了所携带的干扰弹。最终，通过汇总五架无人机共计 **10 枚干扰弹** 产生的遮蔽效果，实现了对三个导弹的 **31.5572 秒的总有效遮蔽时长**。

将各无人机所投放的烟幕干扰弹形成的有效遮蔽时间区间可视化，得到图 20。

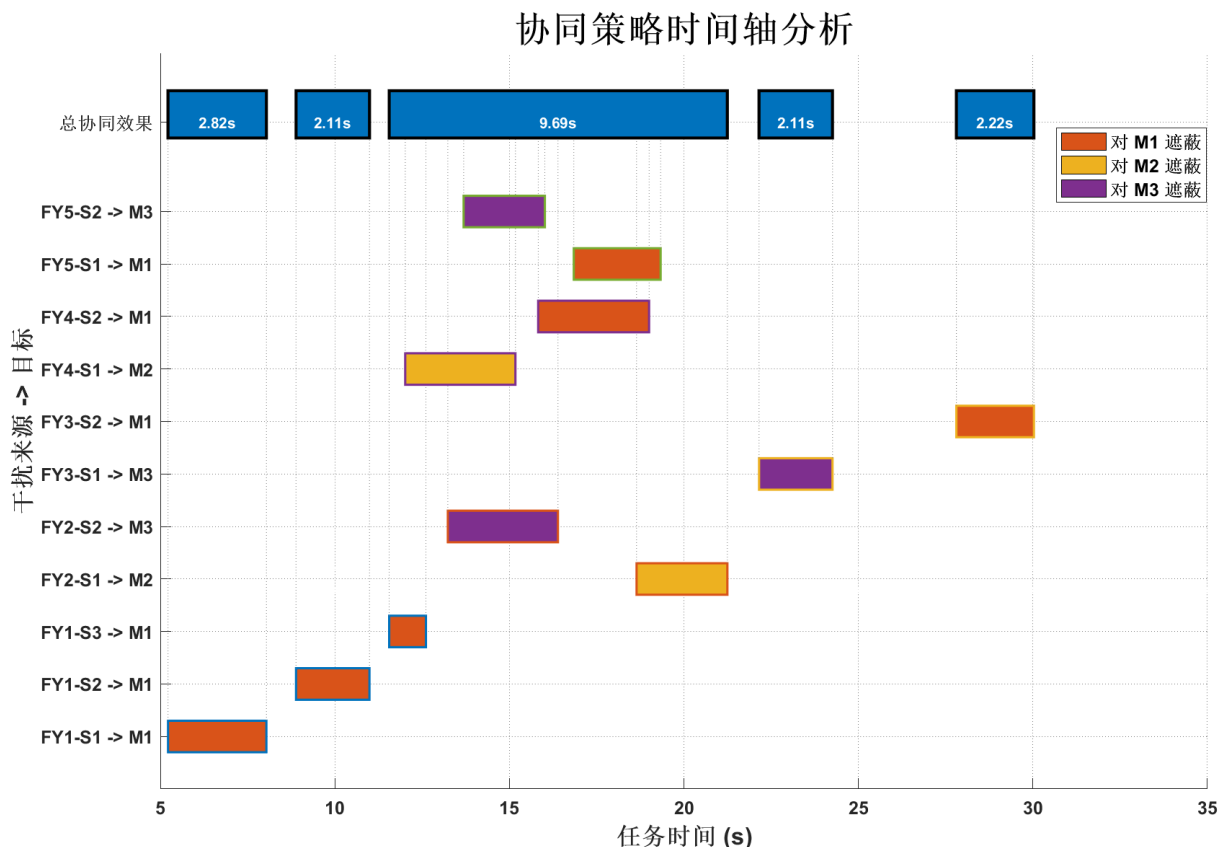


图 20 问题五结果

图中直观地展示所求出的最优协同策略在时间维度上的遮蔽情况与整体效果。总计 31.5572 秒的有效时长并非一次性形成，而是通过各无人机精准的协同配合，构建出多个连续的遮蔽波次。顶部蓝色的总协同效果条展示了整个对 3 个导弹的干扰过程形成的三个主要的有效遮蔽窗口，其中包含了一个长达 9.69 秒的、由多枚干扰弹无缝衔接构成的核心阶段。然而，图中也体现了几个较短的暴露间隙，这说明在极限复杂的对抗中，实现 100% 的全程无缝遮蔽极具挑战性，最优策略是在最大化遮蔽时长的同时，将不可避免的暴露期压缩到最短。

其中，各无人机的飞行参数与投放干扰弹数目汇总得到表 10 所示。分析表 10 可知，所有无人机均采用高速机动战术，飞行速度趋近 140m/s 的性能上限，以快速抵达最佳投放窗口。其中 FY1 作为核心主力使用了全部 3 枚弹药，而其余无人机各使用 2 枚形成支援。最后，各机飞行方向各不相同且经过精确优化，表明策略并非简单趋向目标，而是综合考虑初始位置等因素的复杂解。

表 10 问题五求解的最佳协同策略参数

分析对象	参数	数值
无人机 FY1	飞行速度	140.0000m/s
	飞行方向	179.6432°
	投放干扰弹数量	3 个
无人机 FY2	飞行速度	129.1593m/s
	飞行方向	259.0377°
	投放干扰弹数量	3 个
无人机 FY3	飞行速度	139.9768m/s
	飞行方向	97.4719°
	投放干扰弹数量	2 个
无人机 FY4	飞行速度	139.2204m/s
	飞行方向	302.8788°
	投放干扰弹数量	1 个
无人机 FY5	飞行速度	138.1687m/s
	飞行方向	106.6890°
	投放干扰弹数量	1 个

9.4 模型检验

在建模与求解部分中，所求解出的最优协同策略是基于所有无人机均能从理想的初始位置出发的前提。然而在真实的战场环境中，由于定位系统误差、起飞平台偏移或突发情况等因素，无人机的实际初始位置往往会与预设位置存在一定的随机偏差。因此，检验所制定最优策略在面对这种初始位置不确定性时的表现，即评估策略的鲁棒性，对于衡量其在实战中的可靠性至关重要。

通过蒙特卡洛仿真的方法，量化评估当无人机的初始位置存在随机扰动时，得到的最优策略性能会受到多大影响。

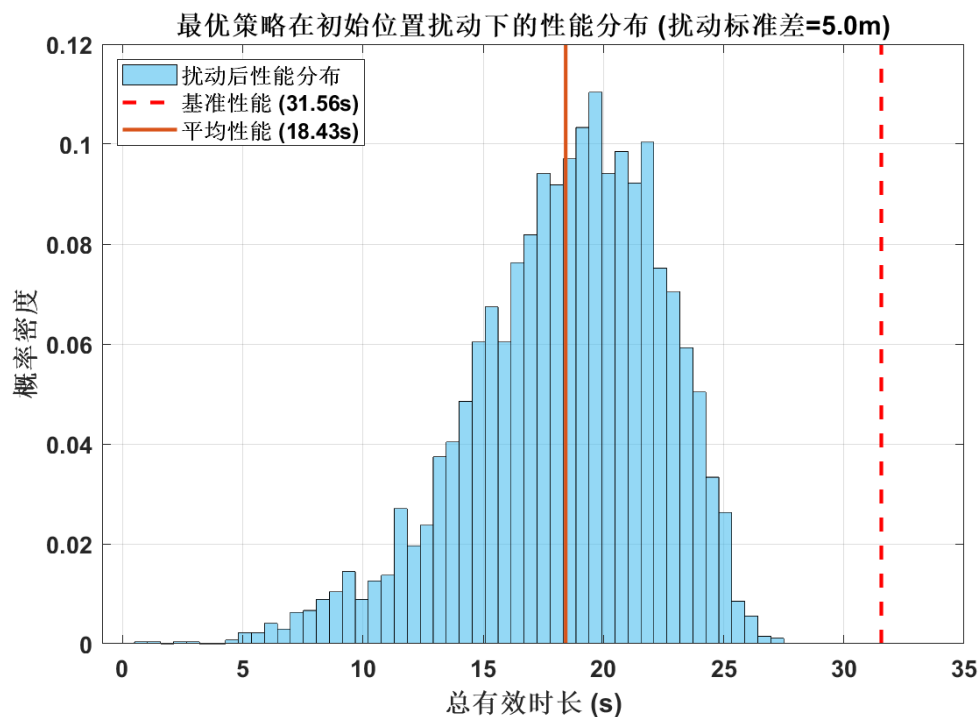


图 21 初始位置扰动影响性能分布图

对于模型检验，采用对每一个无人机初始位置在横纵坐标方向 5m 内加入 5000 次蒙特卡洛随机扰动，更改无人机初始位置，得到扰动后新能分布图 21。分析图 21 可知，引入随机扰动后，模型性能下降了 18.43s，损失百分比为 41.60%，这体现模型对无人机初始位置状态较敏感，同时能维持一定的鲁棒性。

9.5 问题五小结

针对问题五提出的五架无人机协同拦截三枚导弹的复杂场景，将其抽象为一个包含 40 个决策变量的高维度、多约束组合优化问题。在求解策略上，将其分解为 5 个独立 8 变量优化问题，采用基于问题三的简化思路：以为每架无人机规划其三次投放的最优策略为子问题，利用在之前问题中验证有效的遗传算法，为五架无人机并行求解出各自的潜在最优干扰方案；然后通过全局评估与组合，筛选并整合出对三枚来袭导弹总有效遮蔽时长贡献最大的最终协同方案。得到仿真求解结果显示，该最优协同策略能够实现 31.5572 秒的总有效遮蔽时长。进一步进行模型的鲁棒性检验，对每一架无人机的初始位置引入蒙特卡洛随机扰动，策略的平均性能骤降 41.60% 体现了模型具有一定鲁棒性。

十、模型的评价与推广

10.1 模型的优点

结构层次清晰，逻辑递进性强。从确定性系统入手，建立了各实体运动学和有效遮蔽判定的基本框架。逐步增加复杂性，优化单无人机策略、处理单无人机多次投放干扰弹、协调多个独立无人机投放策略，最后分析最复杂的多无人机投放多个干扰弹的情况。由简入繁的构建与推进模型，每一部分都为后续更复杂的问题奠定了坚实基础。

针对不同问题的要求与优化维度进行合适的优化算法选择。对于决策变量数为 4 个的问题二，选择模拟退火法，有效跳出局部最优，寻找全局最优解。对于决策变量增加至 8 个的问题三，采用遗传算法进行处理更高维度搜索空间下的寻优。面对有 12 个变量的问题四，是更高维度优化问题，选用引入量子计算思想的量子遗传算法，利用量子比特的叠加性，增强了种群多样性和全局搜索能力。

10.2 模型的缺点

物理模型过于理想化。在模型中忽略空气阻力，对于抛物线段仅考虑重力，这与真实弹道有较大偏差。并且假设无人机在投放烟幕干扰弹的瞬间，其自身运动状态和干扰弹的初始状态可以瞬时完成，忽略了分离过程的复杂动态。

10.3 模型的改进

深化物理模型，引入空气阻力。在运动微分方程中加入空气阻力项。运动方程会由此变为非线性的常微分方程组，需要用数值方法求解，但能提高轨迹预测的准确性。

10.4 模型的推广

在真实的作战环境中，目标的运动轨迹、无人机的飞行参数以及环境因素都存在不确定性与随机扰动。其目标不再是寻求理想条件下的最长拦截窗口，而是寻找一组在一定扰动范围内仍能保证较高拦截成功率的发射策略，即最大化拦截窗口时长的期望值或最小化脱靶概率。

此外，可以将该优化模型嵌入到战场的指挥控制系统中，用于弹道修正与动态任务规划。当雷达等传感器实时更新了目标的位置和速度信息，发现其偏离预测弹道时，系统可以利用该模型快速重新计算最优拦截方案，实现自适应动态控制，极大提升对高机动目标的拦截能力。

参考文献

- [1] 顾文慧, 高红杰. 烟幕投放控制分析与设计[J]. 光电技术应用, 2014, 29:87-89.
- [2] 陶梦高, 董文锋, 罗威, 等. 空中烟幕投放控制建模与仿真分析[J/OL]. 空天预警研究学报, 2022, 36:281-285. DOI: 10.3969/j.issn.2097-180X.2022.04.010.
- [3] 刘松涛, 姜宁, 王龙涛. 舰载烟幕遮蔽干扰效果在线评估方法研究[J/OL]. 激光与红外, 2016, 46:985-988. DOI: 10.3969/j.issn.1001-5078.2016.08.016.
- [4] 宁聪, 范菁, 孙书超. 多无人机协同规划研究综述[J/OL]. 计算机工程与应用, 2025, 61: 42-58. DOI: 10.3778/j.issn.1002-8331.2405-0110.
- [5] 代瑞恒, 杨先海, 孙阳, 等. 基于改进量子遗传算法的机器人关节轨迹优化[J/OL]. 机床与液压, 2022, 50:6-10. DOI: 10.3969/j.issn.1001-3881.2022.05.002.
- [6] WANG X, TAN G, DAI Y, et al. An optimal guidance strategy for moving-target interception by a multirotor unmanned aerial vehicle swarm[J/OL]. IEEE Access, 2020, 8: 121650-121664. DOI: 10.1109/ACCESS.2020.3006479.
- [7] 华聪聪, 冯胜, 李大华, 等. 基于改进量子遗传算法的光伏智能楼宇负荷优化[J/OL]. 计算机应用与软件, 2024, 41:77-82. DOI: 10.3969/j.issn.1000-386x.2024.09.012.
- [8] 罗笛, 武志东, 凌佳玉. 基于改进量子遗传算法的远程鱼雷航迹规划[J]. 计算机仿真, 2025, 42:12-15.
- [9] SHARMA A, SHOVAL S, SHARMA A, et al. Path planning for multiple targets interception by the swarm of uavs based on swarm intelligence algorithms: A review[J/OL]. IETE Technical Review, 2022, 39(3):675-697. DOI: 10.1080/02564602.2021.1894250.
- [10] GoogleGemini (May 2024version,GeminiPro)

附录 A 文件列表

文件名	功能描述
solve1.m	问题一主程序
solve2.m	问题二主程序
solve3.m	问题三主程序
solve4.m	问题四主程序
solve5.m	问题五主程序
result1.xlsl	问题三结果保存文件
result2.xlsl	问题四结果保存文件
result3.xlsl	问题五结果保存文件

附录 B 代码

solve1.m

```
1
2 % clc; clear; close all;
3
4 fprintf('1. 正在解析题目参数...\n');
5 target.r = 7; target.h = 10; target.bottom = [0, 200, 0];
6 missile.p0 = [20000, 0, 2000]; missile.v = 300;
7 uav.p0 = [17800, 0, 1800]; uav.v = 120;
8 fake_target = [0, 0, 0];
9 missile.tar = fake_target; uav.tar = fake_target;
10 g = [0, 0, -9.8]; t_drop = 1.5; t_fly = 3.6;
11 cloud.r = 10; cloud.vs = 3; cloud.dur = 20;
12
13 fprintf('2. 正在构建运动模型...\n');
14 du = uav.tar - uav.p0; du(3) = 0; uav.vel = uav.v * du /
norm(du);
15 dm = missile.tar - missile.p0; missile.vel = missile.v *
dm / norm(dm);
16 cloud.vel = [0, 0, -cloud.vs];
17 p_drop = uav.p0 + uav.vel * t_drop;
18 cloud.p0 = p_drop + uav.vel * t_fly + 0.5 * g * t_fly^2;
```

```

19     t_deton = t_drop + t_fly;
20     t_start = t_deton; t_end = t_deton + cloud.dur;
21
22     Nc = 100; Ni = 25; dt = 0.5;
23     [theta_c, pts_c] = gen_cyl_pts(Nc, target.r, target.bottom
, target.h);
24
25     fprintf('\n3. 开始循环测试不同插入点数量(Nr)的影响...\n');
26
27     Nr_values = 1:100;
28     total_time_results = zeros(size(Nr_values));
29
30
31     parfor i = 1:length(Nr_values)
32         Nr_current = Nr_values(i);
33
34
35         fprintf('  正在计算 Nr = %d...\n', Nr_current);
36
37         % 调用核心函数进行计算
38         total_time_results(i) = calc_camouflage_time(...
39             t_start, t_end, dt, Nc, Nr_current, Ni, theta_c,
pts_c, ...
40             missile, cloud, t_deton);
41     end
42
43     fprintf('\n--- 所有计算完成 ---\n');
44
45     %% 4. 结果可视化
46     fprintf('4. 正在绘制结果折线图...\n');
47
48     figure('Name', '插入点数量对计算结果的影响', 'NumberTitle'
, 'off');
49     plot(Nr_values, total_time_results, '-o', 'LineWidth',
1.5, 'MarkerSize', 4, 'MarkerFaceColor', 'b');

```

```

50
51     title('精细化搜索插入点数量对总遮蔽时长结果的影响', '
FontSize', 14);
52     xlabel('插入点数量', 'FontSize', 12);
53     ylabel('总遮蔽时长 (s)', 'FontSize', 12);
54     grid on;
55     axis tight;
56
57
58     stable_value = total_time_results(end);
59     fprintf('\n当 Nr = %d 时, 计算结果为 %.40f s\n', Nr_values
(end), stable_value);
60     fprintf('该值可视为一个相对稳定的精确解。 \n');
61
62
63
64
65 %% 核心函数 (以下函数与您提供的代码完全相同, 无需修改)
66 function total = calc_camouflage_time(t_start, t_end, dt, Nc,
Nr, Ni, theta_c, pts_c, missile, cloud, t_det)
67     total = 0;
68     t = t_start;
69     [shaded, unshaded_prev] = check_shade(t, pts_c, missile,
cloud, t_det);
70     was_shaded = shaded;
71     t_start_precise = NaN;
72     if was_shaded, t_start_precise = t_start; end
73
74     while t < t_end
75         t_prev = t;
76         t = t + dt;
77         if t > t_end, t = t_end; end
78
79         [shaded, critical] = check_shade(t, pts_c, missile,
cloud, t_det);

```

```

80
81     if ~was_shaded && shaded
82         t_start_precise = find_trans_time(...
83             t_prev, t, true, theta_c, Nc, Nr, Ni, missile,
84             cloud, t_det, unshaded_prev);
85     elseif was_shaded && ~shaded
86         t_end_precise = find_trans_time(...
87             t_prev, t, false, theta_c, Nc, Nr, Ni, missile
88             , cloud, t_det, critical);
89         if ~isnan(t_start_precise)
90             total = total + (t_end_precise -
91             t_start_precise);
92         end
93         t_start_precise = NaN;
94     end
95     was_shaded = shaded;
96     if ~was_shaded, unshaded_prev = critical; end
97 end
98
99 if was_shaded && ~isnan(t_start_precise)
100     total = total + (t_end - t_start_precise);
101 end
102
103 function [shaded, critical] = check_shade(t, t_pts, missile,
104     cloud, t_det)
105     m_pos = missile.p0 + missile.vel * t;
106     c_pos = cloud.p0 + cloud.vel * (t - t_det);
107     N = size(t_pts, 1) / 2;
108     unshaded = [];
109     for i = 1:size(t_pts, 1)
110         p = t_pts(i, :);
111         d = dist_to_line(c_pos, m_pos, p);
112         if d > cloud.r
113             idx = mod(i-1, N) + 1;

```

```

111         unshaded(end+1) = idx;
112     end
113 end
114 shaded = isempty(unshaded);
115 if shaded, critical = NaN; else, critical = unique(
unshaded); end
116 end
117
118 function t_precise = find_trans_time(t_low, t_high, find_start
    , theta_c, Nc, Nr, Ni, missile, cloud, t_det, critical)
119     target.r = 7; target.h = 10; target.bottom = [0, 200, 0];
120     theta_refine = [];
121     idx_unique = unique(critical);
122     if isempty(idx_unique) || any(isnan(idx_unique)),
idx_unique = 1:Nc; end
123
124     for i = 1:length(idx_unique)
125         c_idx = idx_unique(i);
126         p_idx = mod(c_idx - 2, Nc) + 1;
127         n_idx = mod(c_idx, Nc) + 1;
128         t_start_seg = theta_c(p_idx); t_end_seg = theta_c(
n_idx);
129         if abs(t_end_seg - t_start_seg) > pi
130             if t_end_seg < t_start_seg, t_end_seg = t_end_seg
+ 2*pi;
131             else, t_start_seg = t_start_seg + 2*pi; end
132         end
133         theta_arc = linspace(t_start_seg, t_end_seg, Nr*2);
134         theta_refine = [theta_refine, theta_arc];
135     end
136     theta_refine = unique(theta_refine)';
137     [~, r_pts] = gen_cyl_pts(length(theta_refine), target.r,
target.bottom, target.h, theta_refine);
138
139     for i = 1:Ni

```

```

140         t_mid = (t_low + t_high) / 2;
141         [shaded, ~] = check_shade(t_mid, r_pts, missile, cloud
, t_det);
142         if find_start
143             if shaded, t_high = t_mid; else, t_low = t_mid;
end
144         else
145             if shaded, t_low = t_mid; else, t_high = t_mid;
end
146         end
147     end
148     if find_start, t_precise = t_high; else, t_precise = t_low
; end
149 end
150
151 function [theta, pts] = gen_cyl_pts(N, r, bottom, h, theta_in)
152     if nargin < 5, theta = linspace(0, 2*pi, N+1)'; theta =
theta(1:end-1);
153     else, theta = theta_in; end
154     x = r * cos(theta); y = r * sin(theta); z = zeros(length(
theta), 1);
155     bottom_pts = bottom + [x, y, z];
156     top_pts = bottom + [x, y, z + h];
157     pts = [bottom_pts; top_pts];
158 end
159
160 function d = dist_to_line(p0, p1, p2)
161     v = p2 - p1; w = p0 - p1; c1 = dot(w, v);
162     if c1 <= 0, d = norm(p0 - p1); return; end
163     c2 = dot(v, v);
164     if c2 <= c1, d = norm(p0 - p2); return; end
165     b = c1 / c2; pb = p1 + b * v; d = norm(p0 - pb);
166 end

```

solve2.m

```

1 function solve2()
2     clc; clear; close all;
3     fprintf('--- 开始求解A题 【问题2】 (模拟退火算法) ---\n\n');
4
5     % 模拟退火参数设置
6     T_max = 1000;          % 初始温度
7     T_min = 1e-4;          % 终止温度
8     alpha = 0.99;          % 降温速率
9     iter_L = 300;          % 每个温度迭代次数
10
11     % 决策变量边界 [无人机速度, 飞行方向, 投放延迟, 引信时间]
12     lb = [70, 0, 0.001, 0.001];
13     ub = [140, 2*pi, 30, 30];
14
15     % 初始化
16     current_X = lb + (ub - lb) .* rand(1, 4);
17     current_T = calculate_total_time(current_X);
18     current_E = -current_T;
19
20     best_X = current_X;
21     best_E = current_E;
22
23     history_T = [];
24     history_best_T = [];
25
26     Temp = T_max;
27     iter_count = 0;
28
29     fprintf('开始退火过程...\n');
30
31     % 模拟退火主循环
32     while Temp > T_min
33         for i = 1:iter_L
34             % 生成新解
35             new_X = current_X;

```

```

36         dim_to_change = randi(4);
37         perturbation = (ub(dim_to_change) - lb(
dim_to_change)) * (rand - 0.5) * (Temp / T_max);
38         new_X(dim_to_change) = new_X(dim_to_change) +
perturbation;
39
40         % 边界检查
41         new_X(dim_to_change) = max(lb(dim_to_change), min(
ub(dim_to_change), new_X(dim_to_change)));
42
43         % 计算新解能量
44         new_T = calculate_total_time(new_X);
45         new_E = -new_T;
46
47         % Metropolis准则
48         delta_E = new_E - current_E;
49
50         if delta_E < 0
51             current_X = new_X;
52             current_E = new_E;
53         else
54             p_accept = exp(-delta_E / Temp);
55             if rand() < p_accept
56                 current_X = new_X;
57                 current_E = new_E;
58             end
59         end
60
61         % 更新最优解
62         if current_E < best_E
63             best_E = current_E;
64             best_X = current_X;
65         end
66         iter_count = iter_count + 1;
67     end

```

```

68
69     % 记录数据
70     history_T(end+1) = -current_E;
71     history_best_T(end+1) = -best_E;
72
73     % 降温
74     Temp = Temp * alpha;
75     fprintf('温度: %.4f, 当前最优时长: %.6f s\n', Temp, -
best_E);
76     end
77
78     % 结果展示
79     fprintf('\n--- 模拟退火完成 ---\n');
80     optimal_T = -best_E;
81     optimal_v_uav = best_X(1);
82     optimal_theta_uav = best_X(2);
83     optimal_t_drop = best_X(3);
84     optimal_t_flight = best_X(4);
85
86     fprintf('最大有效遮蔽时长: %.8f s\n', optimal_T);
87     fprintf(' - 无人机飞行速度: %.4f m/s\n', optimal_v_uav);
88     fprintf(' - 无人机飞行方向(弧度): %.4f (角度: %.2f°)\n',
optimal_theta_uav, rad2deg(optimal_theta_uav));
89     fprintf(' - 投放延迟时间: %.4f s\n', optimal_t_drop);
90     fprintf(' - 烟幕弹引信时间: %.4f s\n', optimal_t_flight);
91
92     % 计算投放点和起爆点
93     uav.p0 = [17800, 0, 1800];
94     g = [0, 0, -9.8];
95     uav.vel = [optimal_v_uav * cos(optimal_theta_uav),
optimal_v_uav * sin(optimal_theta_uav), 0];
96     p_drop = uav.p0 + uav.vel * optimal_t_drop;
97     p_deton = p_drop + uav.vel * optimal_t_flight + 0.5 * g *
optimal_t_flight.^2;
98

```

```

99     fprintf(' - 最佳投放点: [%.2f, %.2f, %.2f]\n', p_drop);
100    fprintf(' - 最佳起爆点: [%.2f, %.2f, %.2f]\n', p_deton);
101
102    % 绘制收敛曲线
103    figure('Name', '模拟退火收敛过程');
104    plot(history_best_T, 'r-', 'LineWidth', 2);
105    hold on;
106    plot(history_T, 'b--', 'LineWidth', 1);
107    legend('历史最优时长', '当前时长', 'Location', 'southeast'
108    );
109    title('模拟退火算法收敛曲线', 'FontSize', 14);
110    xlabel('降温次数', 'FontSize', 12);
111    ylabel('有效遮蔽时长 (s)', 'FontSize', 12);
112    grid on;
113 end
114 % 计算总遮蔽时长
115 function total_time = calculate_total_time(X)
116     % 解码决策变量
117     v_uav      = X(1);
118     theta_uav  = X(2);
119     t_drop     = X(3);
120     t_flight   = X(4);
121
122     % 物理参数
123     target.r = 7; target.h = 10; target.bottom = [0, 200, 0];
124     missile.p0 = [20000, 0, 2000]; missile.v = 300;
125     uav.p0 = [17800, 0, 1800];
126     fake_target = [0, 0, 0];
127     missile.tar = fake_target;
128     g = [0, 0, -9.8];
129     cloud.r = 10; cloud.vs = 3; cloud.dur = 20;
130
131     % 运动模型
132     uav.vel = [v_uav * cos(theta_uav), v_uav * sin(theta_uav),

```

```

0];
133     dm = missile.tar - missile.p0;
134     missile.vel = missile.v * dm / norm(dm);
135
136     cloud.vel = [0, 0, -cloud.vs];
137     p_drop = uav.p0 + uav.vel * t_drop;
138     cloud.p0 = p_drop + uav.vel * t_flight + 0.5 * g *
t_flight.^2;
139
140     % 物理约束检查
141     if cloud.p0(3) < 0
142         total_time = 0;
143         return;
144     end
145
146     t_deton = t_drop + t_flight;
147     t_start = t_deton;
148     t_end = t_deton + cloud.dur;
149
150     % 仿真参数
151     Nc = 50; Nr = 50; Ni = 15; dt = 0.5;
152
153     [theta_c, pts_c] = gen_cyl_pts(Nc, target.r, target.bottom
, target.h);
154
155     total_time = calc_camouflage_time(...
156         t_start, t_end, dt, Nc, Nr, Ni, theta_c, pts_c, ...
157         missile, cloud, t_deton);
158 end
159
160 % 计算总遮蔽时间
161 function total = calc_camouflage_time(t_start, t_end, dt, Nc,
Nr, Ni, theta_c, pts_c, missile, cloud, t_det)
162     total = 0;
163     t = t_start;

```

```

164     [shaded, unshaded_prev] = check_shade(t, pts_c, missile,
cloud, t_det);
165     was_shaded = shaded;
166     t_start_precise = NaN;
167     if was_shaded, t_start_precise = t_start; end
168
169     while t < t_end
170         t_prev = t;
171         t = t + dt;
172         if t > t_end, t = t_end; end
173
174         [shaded, critical] = check_shade(t, pts_c, missile,
cloud, t_det);
175
176         % 状态转变处理
177         if ~was_shaded && shaded
178             t_start_precise = find_trans_time(...
179                 t_prev, t, true, theta_c, Nc, Nr, Ni, missile,
cloud, t_det, unshaded_prev);
180         elseif was_shaded && ~shaded
181             t_end_precise = find_trans_time(...
182                 t_prev, t, false, theta_c, Nc, Nr, Ni, missile
, cloud, t_det, critical);
183             if ~isnan(t_start_precise)
184                 total = total + (t_end_precise -
t_start_precise);
185             end
186             t_start_precise = NaN;
187         end
188
189         was_shaded = shaded;
190         if ~was_shaded, unshaded_prev = critical; end
191     end
192
193     % 处理结束时仍遮蔽的情况

```

```

194     if was_shaded && ~isnan(t_start_precise)
195         total = total + (t_end - t_start_precise);
196     end
197 end
198
199 % 检查某一时刻是否完全遮蔽
200 function [shaded, critical] = check_shade(t, t_pts, missile,
    cloud, t_det)
201     m_pos = missile.p0 + missile.vel * t;
202     c_pos = cloud.p0 + cloud.vel * (t - t_det);
203
204     N = size(t_pts, 1) / 2;
205     unshaded = [];
206
207     for i = 1:size(t_pts, 1)
208         p = t_pts(i, :);
209         d = dist_to_line(c_pos, m_pos, p);
210         if d > cloud.r
211             idx = mod(i-1, N) + 1;
212             unshaded(end+1) = idx;
213         end
214     end
215
216     shaded = isempty(unshaded);
217     if shaded
218         critical = NaN;
219     else
220         critical = unique(unshaded);
221     end
222 end
223
224 % 二分法查找精确的状态转变时间
225 function t_precise = find_trans_time(t_low, t_high, find_start
    , theta_c, Nc, Nr, Ni, missile, cloud, t_det, critical)
226     target.r = 7; target.h = 10; target.bottom = [0, 200, 0];

```

```

227     theta_refine = [];
228     idx_unique = unique(critical);
229
230     if isempty(idx_unique) || any(isnan(idx_unique)),
idx_unique = 1:Nc; end
231
232     % 生成精细角度点
233     for i = 1:length(idx_unique)
234         c_idx = idx_unique(i);
235         p_idx = mod(c_idx - 2, Nc) + 1;
236         n_idx = mod(c_idx, Nc) + 1;
237
238         t_start_seg = theta_c(p_idx);
239         t_end_seg = theta_c(n_idx);
240
241         % 处理角度跨0点情况
242         if abs(t_end_seg - t_start_seg) > pi
243             if t_end_seg < t_start_seg
244                 t_end_seg = t_end_seg + 2*pi;
245             else
246                 t_start_seg = t_start_seg + 2*pi;
247             end
248         end
249
250         theta_arc = linspace(t_start_seg, t_end_seg, Nr*2);
251         theta_refine = [theta_refine, theta_arc];
252     end
253
254     theta_refine = unique(theta_refine)';
255     [~, r_pts] = gen_cyl_pts(length(theta_refine), target.r,
target.bottom, target.h, theta_refine);
256
257     % 二分法精确查找
258     for i = 1:Ni
259         t_mid = (t_low + t_high) / 2;

```

```

260         [shaded, ~] = check_shade(t_mid, r_pts, missile, cloud
    , t_det);
261
262         if find_start
263             if shaded
264                 t_high = t_mid;
265             else
266                 t_low = t_mid;
267             end
268         else
269             if shaded
270                 t_low = t_mid;
271             else
272                 t_high = t_mid;
273             end
274         end
275     end
276
277     if find_start
278         t_precise = t_high;
279     else
280         t_precise = t_low;
281     end
282 end
283
284 % 生成圆柱轮廓采样点
285 function [theta, pts] = gen_cyl_pts(N, r, bottom, h, theta_in)
286     if nargin < 5
287         theta = linspace(0, 2*pi, N+1)';
288         theta = theta(1:end-1);
289     else
290         theta = theta_in;
291     end
292
293     x = r * cos(theta);

```

```

294     y = r * sin(theta);
295     z = zeros(length(theta), 1);
296
297     bottom_pts = bottom + [x, y, z];
298     top_pts = bottom + [x, y, z + h];
299     pts = [bottom_pts; top_pts];
300 end
301
302 % 计算点到线段的距离
303 function d = dist_to_line(p0, p1, p2)
304     v = p2 - p1;
305     w = p0 - p1;
306     c1 = dot(w, v);
307
308     if c1 <= 0
309         d = norm(p0 - p1);
310         return;
311     end
312
313     c2 = dot(v, v);
314     if c2 <= c1
315         d = norm(p0 - p2);
316         return;
317     end
318
319     b = c1 / c2;
320     pb = p1 + b * v;
321     d = norm(p0 - pb);
322 end

```

solve3.m

```

1 function solve3()
2     clear; clc; close all;
3     fprintf('--- 运行混合遗传算法 ---\n');
4

```

```

5     params.g = 9.8;
6     params.M1_pos0 = [20000, 0, 2000];
7     params.M1_vel_scalar = 300;
8     M1_target_pos = [0, 0, 0];
9     params.M1_vel_vec = (M1_target_pos - params.M1_pos0) /
norm(M1_target_pos - params.M1_pos0) * params.M1_vel_scalar;
10    params.M2_center_bottom = [0, 200, 0];
11    params.M2_r = 7;
12    params.M2_h = 10;
13    params.uav_pos0 = [17800, 0, 1800];
14    uav_alt = params.uav_pos0(3);
15    vec_to_target = params.M2_center_bottom(1:2) - params.
uav_pos0(1:2);
16    params.theta_base = atan2(vec_to_target(2), vec_to_target
(1));
17    params.smoke_r = 10;
18    params.smoke_sink_v = 3;
19    params.smoke_dur = 20;
20    N_target = 30;
21    [~, params.target_pts] = generate_cyl_pts(N_target, params
.M2_r, params.M2_center_bottom, params.M2_h);
22
23    pop_size = 200;
24    max_gen = 3000;
25    elite_num = 10;
26    mut_rate = 0.5;
27    cross_rate = 0.8;
28
29    lb = [70, -pi/4, 0.1, 0.1, 1.0, 0.1, 1.0, 0.1];
30    T_flight_max_phy = sqrt(2 * uav_alt / params.g);
31    T_mission_max = norm(params.M1_pos0) / params.
M1_vel_scalar - 5;
32    delta_t_max = 25;
33    ub = [140, pi/4, T_mission_max, T_flight_max_phy, ...
34          delta_t_max, T_flight_max_phy, ...

```

```

35         delta_t_max,    T_flight_max_phy];
36
37     init_pop = zeros(pop_size, length(lb));
38     x_elite = [
39         140, deg2rad(179.64) - params.theta_base, 0.10, 3.78,
40         3.26, 5.30, 5.42, 5.98
41     ];
42     init_pop(1, :) = repair(x_elite, lb, ub);
43
44     for i = 2:pop_size
45         init_pop(i, :) = mutate_elite(init_pop(1, :), lb, ub,
1.0);
46     end
47
48     pop = init_pop;
49     hist_best = zeros(max_gen, 1);
50     stag_cnt = 0;
51     last_best = -realmax;
52     fprintf('\n开始运行混合遗传算法...\n');
53     if isempty(gcp('nocreate')), parpool(); end
54     for gen = 1:max_gen
55         fitness = zeros(pop_size, 1);
56         parfor i = 1:pop_size
57             fitness(i) = obj(pop(i, :), params);
58         end
59
60         [sorted_fit, sorted_idx] = sort(fitness);
61         hist_best(gen) = -sorted_fit(1);
62
63         if abs(hist_best(gen) - last_best) < 1e-6
64             stag_cnt = stag_cnt + 1;
65         else
66             stag_cnt = 0;
67             last_best = hist_best(gen);
68         end

```

```

69         if stag_cnt >= 50
70             fprintf('\n>> 算法停滞 %d 代，触发灾变! \n',
stag_cnt);
71
72             sorted_pop = pop(sorted_idx, :);
73             elites = sorted_pop(1:elite_num, :);
74
75             lb_tight = [70, -pi/6, 0.1, 0.1, 1.0, 0.1, 1.0,
0.1];
76             ub_tight = [140, pi/6, 20, 15, 20, 15, 20,
15];
77             new_num = pop_size - elite_num;
78             new_blood = lb_tight + (ub_tight - lb_tight) .*
rand(new_num, length(lb));
79
80             pop = [elites; new_blood];
81             stag_cnt = 0;
82             last_best = -realmax;
83             fprintf('>> 灾变完成，进入下一代。 \n\n');
84             continue;
85         end
86
87         sorted_pop = pop(sorted_idx, :);
88         next_pop = zeros(size(pop));
89         next_pop(1:elite_num, :) = sorted_pop(1:elite_num, :);
90
91         for i = (elite_num + 1):pop_size
92             p1_idx = tour_select(pop_size);
93             p2_idx = tour_select(pop_size);
94             p1 = sorted_pop(p1_idx, :);
95             p2 = sorted_pop(p2_idx, :);
96
97             if rand() < cross_rate
98                 child = cross(p1, p2);
99             else

```

```

100         child = p1;
101     end
102
103     if rand() < mut_rate
104         mut_scale = 1 - (gen / max_gen)^2;
105         child = mutate_elite(child, lb, ub, mut_scale)
106     ;
107
108     end
109
110     next_pop(i, :) = repair(child, lb, ub);
111 end
112
113 pop = next_pop;
114
115 if mod(gen, 10) == 0 || gen == 1
116     fprintf('代数: %d / %d, 最佳时长: %.6f s\n', gen,
max_gen, hist_best(gen));
117 end
118 end
119
120 [final_best, final_idx] = min(fitness);
121 x_opt = pop(final_idx, :);
122 max_t = -final_best;
123
124 figure; plot(hist_best, 'LineWidth', 2);
125 title('GA收敛过程'); xlabel('代数'); ylabel('最佳时长 (s)'
); grid on;
126
127 v_opt = x_opt(1);
128 d_theta_opt = x_opt(2);
129 theta_opt = params.theta_base + d_theta_opt;
130 t_drop1_opt = x_opt(3);
131 d_t_12_opt = x_opt(5);
132 d_t_23_opt = x_opt(7);
133 t_drop2_opt = t_drop1_opt + d_t_12_opt;

```

```

132     t_drop3_opt = t_drop2_opt + d_t_23_opt;
133
134     fprintf('\n--- 最佳策略 ---\n');
135     fprintf('最大有效时长: %.6f s\n', max_t);
136     dir_deg = rad2deg(theta_opt);
137     if dir_deg < 0, dir_deg = dir_deg + 360; end
138     fprintf(' - 速度: %.4f m/s, 方向: %.2f°\n', v_opt,
dir_deg);
139     fprintf(' - 弹1: 投放延迟 %.2fs, 引信时间 %.2fs\n',
t_drop1_opt, x_opt(4));
140     fprintf(' - 弹2: 投放延迟 %.2fs (间隔 %.2fs), 引信时间
%.2fs\n', t_drop2_opt, d_t_12_opt, x_opt(6));
141     fprintf(' - 弹3: 投放延迟 %.2fs (间隔 %.2fs), 引信时间
%.2fs\n', t_drop3_opt, d_t_23_opt, x_opt(8));
142 end
143
144 function child = cross(p1, p2)
145     alpha = 0.5;
146     diff = abs(p1 - p2);
147     lower = min(p1, p2) - alpha * diff;
148     upper = max(p1, p2) + alpha * diff;
149     child = lower + (upper - lower) .* rand(size(p1));
150 end
151
152 function mut_x = mutate_elite(x, lb, ub, scale)
153     mut_x = x;
154     strategy = randi(4);
155     if strategy == 1 && rand() < 0.5
156         mut_x(7) = 1.0 + rand() * 4 * scale;
157         mut_x(8) = x(6) + (rand()-0.5) * 4 * scale;
158     else
159         dims = [1,2,3,4,5,6,8];
160         dim = dims(randi(length(dims)));
161         step_rng = (ub(dim) - lb(dim)) * 0.1 * scale;
162         step = step_rng * (2 * rand() - 1);

```

```

163         mut_x(dim) = x(dim) + step;
164     end
165 end
166
167 function x_rep = repair(x, lb, ub)
168     x_row = reshape(x, 1, []);
169     x_rep = max(lb, min(ub, x_row));
170     if x_rep(5) < 1.0
171         x_rep(5) = 1.0;
172     end
173     if x_rep(7) < 1.0
174         x_rep(7) = 1.0;
175     end
176 end
177
178 function idx = tour_select(pop_size, tour_size)
179     if nargin < 2, tour_size = 3; end
180     candidates = randi(pop_size, 1, tour_size);
181     idx = min(candidates);
182 end
183
184 function neg_t = obj(x, params)
185     v = x(1);
186     d_theta = x(2);
187     t_drop1 = x(3);
188     t_flight1 = x(4);
189     d_t_12 = x(5);
190     t_flight2 = x(6);
191     d_t_23 = x(7);
192     t_flight3 = x(8);
193
194     t_drop2 = t_drop1 + d_t_12;
195     t_drop3 = t_drop2 + d_t_23;
196
197     T_mission_max = norm(params.M1_pos0) / params.

```

```

M1_vel_scalar;
198     if t_drop3 > T_mission_max
199         neg_t = 0;
200         return;
201     end
202
203     theta = params.theta_base + d_theta;
204     t_drops = [t_drop1, t_drop2, t_drop3];
205     t_flights = [t_flight1, t_flight2, t_flight3];
206
207     g = params.g;
208     uav_vel = [v * cos(theta), v * sin(theta), 0];
209
210     all_intervals = [];
211     for i = 1:3
212         t_drop = t_drops(i);
213         t_flight = t_flights(i);
214         pos_drop = params.uav_pos0 + uav_vel * t_drop;
215         pos_deton = pos_drop + uav_vel * t_flight + 0.5 * [0,
0, -g] * t_flight^2;
216         t_deton = t_drop + t_flight;
217
218         intervals_i = calc_camo_intervals(t_deton, pos_deton,
params);
219         if ~isempty(intervals_i)
220             all_intervals = [all_intervals; intervals_i];
221         end
222     end
223
224     total_t = merge_intervals(all_intervals);
225     neg_t = -total_t;
226 end
227 function intervals = calc_camo_intervals(t_deton, pos_deton,
params)
228     intervals = [];

```

```

229     if pos_deton(3) < 0, return; end
230
231     t_start = t_deton;
232     t_end = t_deton + params.smoke_dur;
233
234     N_coarse = 20;
235     t_vec = linspace(t_start, t_end, N_coarse);
236     is_camo = false(1, N_coarse);
237     for i = 1:N_coarse
238         is_camo(i) = is_fully_camo(t_vec(i), t_deton,
pos_deton, params);
239     end
240
241     if ~any(is_camo), return; end
242
243     diff_vec = diff([0, is_camo, 0]);
244     start_idx = find(diff_vec == 1);
245     end_idx = find(diff_vec == -1);
246
247     N_bisect = 15;
248
249     for i = 1:length(start_idx)
250         idx_low = max(1, start_idx(i) - 1);
251         idx_high = min(N_coarse, end_idx(i));
252         t_low = t_vec(idx_low);
253         t_high = t_vec(idx_high);
254
255         t_start_prec = find_prec_bound(t_low, t_high, true,
N_bisect, t_deton, pos_deton, params);
256         t_end_prec = find_prec_bound(t_low, t_high, false,
N_bisect, t_deton, pos_deton, params);
257
258         if t_end_prec > t_start_prec
259             intervals = [intervals; t_start_prec, t_end_prec];
260         end

```

```

261     end
262 end
263 function t_prec = find_prec_bound(t_low, t_high, find_start,
    N_iter, t_det, pos_det, params)
264     for i = 1:N_iter
265         t_mid = (t_low + t_high) / 2;
266         is_mid = is_fully_camo(t_mid, t_det, pos_det, params);
267         if find_start
268             if is_mid, t_high = t_mid; else, t_low = t_mid;
        end
269         else
270             if is_mid, t_low = t_mid; else, t_high = t_mid;
        end
271     end
272 end
273     if find_start, t_prec = t_high; else, t_prec = t_low; end
274 end
275 function total_dur = merge_intervals(intervals)
276     if isempty(intervals), total_dur = 0; return; end
277     sorted = sortrows(intervals, 1);
278     merged = sorted(1,:);
279     for i = 2:size(sorted, 1)
280         curr = sorted(i,:);
281         last_end = merged(end, 2);
282         if curr(1) <= last_end
283             merged(end, 2) = max(last_end, curr(2));
284         else
285             merged = [merged; curr];
286         end
287     end
288     total_dur = sum(merged(:,2) - merged(:,1));
289 end
290 function is_camo = is_fully_camo(t, t_det, pos_det, params)
291     missile_pos = params.M1_pos0 + params.M1_vel_vec * t;
292     cloud_pos = pos_det - [0, 0, params.smoke_sink_v * (t -

```

```

    t_det)];
293     for i = 1:size(params.target_pts, 1)
294         p_tar = params.target_pts(i, :);
295         dist = pt_seg_dist(cloud_pos, missile_pos, p_tar);
296         if dist > params.smoke_r
297             is_camo = false;
298             return;
299         end
300     end
301     is_camo = true;
302 end
303 function dist = pt_seg_dist(p0, p1, p2)
304     v = p2 - p1; w = p0 - p1; c1 = dot(w, v);
305     if c1 <= 0, dist = norm(p0 - p1); return; end
306     c2 = dot(v, v);
307     if c2 <= c1, dist = norm(p0 - p2); return; end
308     b = c1 / c2; pb = p1 + b * v; dist = norm(p0 - pb);
309 end
310 function [theta_out, pts] = generate_cyl_pts(N, r, b_center, h
    , theta_in)
311     if nargin < 5
312         theta_out = linspace(0, 2*pi, N+1)';
313         theta_out = theta_out(1:end-1);
314     else
315         theta_out = theta_in;
316     end
317     x = r * cos(theta_out); y = r * sin(theta_out); z = zeros(
length(theta_out), 1);
318     bottom_pts = b_center + [x, y, z]; top_pts = b_center + [x
    , y, z + h];
319     pts = [bottom_pts; top_pts];
320 end

```

solve4.m

```
1 function solve4()
```

```

2
3 clear; clc; close all;
4 params.g = 9.8;
5 params.M1_pos0 = [20000, 0, 2000];
6 params.M1_vel_scalar = 300;
7 M1_target_pos = [0, 0, 0];
8 params.M1_vel_vec = (M1_target_pos - params.M1_pos0) /
norm(M1_target_pos - params.M1_pos0) * params.M1_vel_scalar;
9 params.M2_center_bottom = [0, 200, 0];
10 params.M2_r = 7;
11 params.M2_h = 10;
12 params.uav_pos0 = [
13     17800, 0, 1800;
14     12000, 1400, 1400;
15     6000, -3000, 700
16 ];
17 params.num_uavs = 3;
18 N_target = 50;
19 params.target_pts = generate_cyl_pts(N_target, params.M2_r
, params.M2_center_bottom, params.M2_h);
20 params.smoke_r = 10;
21 params.smoke_sink_v = 3;
22 params.smoke_dur = 20;
23
24 pop_size = 500;
25 max_gen = 2000;
26 L = 12;
27 theta_min = 0.001 * pi;
28 theta_max = 0.05 * pi;
29 crossover_prob_base = 0.7; % 基础交叉概率
30 crossover_prob_elite = 0.9; % 精英交叉概率
31
32 num_vars_per_uav = 4;
33 num_vars = params.num_uavs * num_vars_per_uav;
34 n_genes = num_vars * L;

```

```

35
36     lb = []; ub = [];
37     T_mission_max = norm(params.M1_pos0) / params.
M1_vel_scalar - 5;
38     for i = 1:params.num_uavs
39         uav_alt = params.uav_pos0(i, 3);
40         T_flight_max_phy = sqrt(2 * uav_alt / params.g);
41         lb_i = [70, -pi, 0.0001, 0.0001];
42         ub_i = [140, pi, T_mission_max, T_flight_max_phy];
43         lb = [lb, lb_i];
44         ub = [ub, ub_i];
45     end
46
47
48     [best_vars, best_fitness, fitness_history] = run_qga();
49
50     max_t = -best_fitness;
51     figure;
52     plot(-fitness_history, 'LineWidth', 2);
53     title('QGA 收敛过程');
54     xlabel('代数');
55     ylabel('最佳总时长 (s)');
56     grid on;
57
58     output_and_save_results(best_vars, max_t, params,
num_vars_per_uav);
59
60
61     function [best_vars, global_best_fitness, fitness_history]
= run_qga()
62         stagnation_threshold = 50;
63         stagnation_counter = 0;
64         reset_proportion = 0.4;
65
66         fitness_history = -inf(max_gen, 1);

```

```

67     global_best_fitness = inf;
68     global_best_solution_binary = [];
69     global_best_q_chromosome = [];
70
71     q_pop = initialize_q_population(pop_size, n_genes);
72
73     fprintf('开始运行量子遗传算法...\n');
74     pool = gcp('nocreate');
75     if isempty(pool), parpool(); end
76
77     for gen = 1:max_gen
78         % 1. 测量与评估
79         p_pop = zeros(pop_size, n_genes);
80         for i = 1:pop_size
81             q_chromosome = q_pop{i};
82             p_pop(i, :) = (rand(1, n_genes) > q_chromosome
(1, :).^2);
83         end
84
85         fitness = zeros(pop_size, 1);
86         parfor i = 1:pop_size
87             continuous_individual = decode_population(
p_pop(i, :), L, num_vars, lb, ub);
88             fitness(i) = obj_q4(continuous_individual,
params);
89         end
90
91         % 2. 更新全局最优解
92         [min_fitness_current_gen, best_idx_current_gen] =
min(fitness);
93         if min_fitness_current_gen < global_best_fitness
94             global_best_fitness = min_fitness_current_gen;
95             global_best_solution_binary = p_pop(
best_idx_current_gen, :);
96             global_best_q_chromosome = q_pop{

```

```

best_idx_current_gen};
97         stagnation_counter = 0;
98     else
99         stagnation_counter = stagnation_counter + 1;
100     end
101     fitness_history(gen) = global_best_fitness;
102
103     % 3. 量子交叉操作
104     f_for_selection = -fitness;
105     f_for_selection = f_for_selection - min(
f_for_selection) + 1e-6;
106     if sum(f_for_selection) < 1e-9, selection_probs =
ones(pop_size,1)/pop_size; else, selection_probs =
f_for_selection / sum(f_for_selection); end
107
108     new_q_pop_from_crossover = q_pop;
109     for i = 1:2:pop_size
110         parent1_idx = select_by_roulette(
selection_probs);
111         parent2_idx = select_by_roulette(
selection_probs);
112
113         f_parent1 = -fitness(parent1_idx);
114         f_parent2 = -fitness(parent2_idx);
115         f_avg = -mean(fitness);
116         f_prime = max(f_parent1, f_parent2);
117
118         crossover_prob = crossover_prob_base;
119         if f_prime > f_avg && ((-global_best_fitness)
- f_avg) > 1e-6
120             crossover_prob = crossover_prob_elite * (
f_prime - f_avg) / ((-global_best_fitness) - f_avg);
121         end
122
123         if rand < crossover_prob

```

```

124         crossover_point = randi(n_genes - 1);
125         child1_q = [q_pop{parent1_idx}(:, 1:
crossover_point), q_pop{parent2_idx}(:, crossover_point+1:
end)];
126         child2_q = [q_pop{parent2_idx}(:, 1:
crossover_point), q_pop{parent1_idx}(:, crossover_point+1:
end)];
127         new_q_pop_from_crossover{i} = child1_q;
128         if i+1 <= pop_size
129             new_q_pop_from_crossover{i+1} =
child2_q;
130         end
131     end
132 end
133 q_pop = new_q_pop_from_crossover;
134
135 % 4. 自适应量子门更新
136 for i = 1:pop_size
137     delta_theta =
get_rotation_params_fitness_based(fitness(i),
global_best_fitness, p_pop(i,:), global_best_solution_binary
);
138     q_chromosome = q_pop{i};
139     for j=1:n_genes
140         if delta_theta(j) ~= 0
141             angle = delta_theta(j);
142             rotation_matrix = [cos(angle), -sin(
angle); sin(angle), cos(angle)];
143             q_chromosome(:,j) = rotation_matrix *
q_chromosome(:,j);
144         end
145     end
146     q_pop{i} = q_chromosome;
147 end
148

```

```

149         % 5. 大灾变与精英保留
150         if stagnation_counter >= stagnation_threshold
151             fprintf('>>> 代数 %d: 检测到停滞，触发大灾变!
152             <<<\n', gen);
153             other_indices = 1:pop_size;
154             other_indices(other_indices ==
best_idx_current_gen) = [];
155             num_to_reset = floor(reset_proportion *
pop_size);
156             indices_to_reset = other_indices(randperm(
length(other_indices), num_to_reset));
157             for i = indices_to_reset
158                 q_pop{i} = repmat([1/sqrt(2); 1/sqrt(2)],
1, n_genes);
159             end
160             stagnation_counter = 0;
161         end
162         [~, worst_idx] = max(fitness);
163         if ~isempty(global_best_q_chromosome)
164             q_pop{worst_idx} = global_best_q_chromosome;
165         end
166
167         if mod(gen, 10) == 0 || gen == 1
168             fprintf('代数: %d / %d, 最佳总时长: %.6f s (停
滞: %d代)\n', gen, max_gen, -fitness_history(gen),
stagnation_counter);
169         end
170     end
171
172     delete(gcp('nocreate'));
173     fprintf('并行池已关闭.\n');
174
175     best_vars = decode_population(
global_best_solution_binary, L, num_vars, lb, ub);

```

```

176     end
177
178     function delta_theta = get_rotation_params_fitness_based(
current_fitness, global_best_fitness, current_binary,
global_best_binary)
179         s = zeros(1, n_genes);
180         if current_fitness > global_best_fitness
181             diff_indices = find(current_binary ~=
global_best_binary);
182             if ~isempty(diff_indices)
183                 s(diff_indices(current_binary(diff_indices)
==0)) = 1;
184                 s(diff_indices(current_binary(diff_indices)
==1)) = -1;
185             end
186         end
187         f_diff = abs(current_fitness - global_best_fitness);
188         max_possible_fitness_diff = 50;
189         f_norm = min(f_diff / max_possible_fitness_diff, 1);
190         adaptive_magnitude = theta_min + (theta_max -
theta_min) * f_norm;
191         delta_theta = s * adaptive_magnitude;
192     end
193
194     function selected_index = select_by_roulette(probabilities
)
195         cumulative_probs = cumsum(probabilities);
196         r = rand();
197         selected_index = find(cumulative_probs >= r, 1, 'first
');
198         if isempty(selected_index), selected_index = length(
probabilities); end
199     end
200 end
201

```

```

202
203
204
205
206 function q_pop = initialize_q_population(pop_size, n_genes)
207     val = 1 / sqrt(2);
208     q_pop = cell(pop_size, 1);
209     for i = 1:pop_size
210         q_pop{i} = repmat([val; val], 1, n_genes);
211     end
212 end
213
214 function continuous_vars = decode_population(p_ind, L,
    num_vars, lb, ub)
215     continuous_vars = zeros(1, num_vars);
216     for k = 1:num_vars
217         start_idx = (k-1)*L + 1;
218         end_idx = k*L;
219         binary_str = p_ind(start_idx:end_idx);
220         decimal_val = bin2dec(num2str(binary_str(:)'));
221         continuous_vars(k) = lb(k) + decimal_val * (ub(k) - lb
(k)) / (2^L - 1);
222     end
223 end
224
225 function neg_t = obj_q4(x, params)
226     all_intervals = [];
227     num_vars_per_uav = 4;
228     for i = 1:params.num_uavs
229         start_idx = (i-1)*num_vars_per_uav+1;
230         v = x(start_idx);
231         theta = x(start_idx+1);
232         t_drop = x(start_idx+2);
233         t_flight = x(start_idx+3);
234

```

```

235         T_max_local = norm(params.M1_pos0) / params.
M1_vel_scalar;
236         if t_drop + t_flight > T_max_local
237             neg_t = 0;
238             return;
239         end
240
241         uav_vel = [v*cos(theta), v*sin(theta), 0];
242         pos_drop = params.uav_pos0(i,:) + uav_vel * t_drop;
243         pos_deton = pos_drop + uav_vel * t_flight + 0.5 * [0,
0, -params.g] * t_flight^2;
244         t_deton = t_drop + t_flight;
245
246         intervals_i = calc_camo_intervals(t_deton, pos_deton,
params);
247         if ~isempty(intervals_i)
248             all_intervals = [all_intervals; intervals_i];
249         end
250     end
251     total_t = merge_intervals(all_intervals);
252     neg_t = -total_t;
253 end
254
255 function output_and_save_results(best_vars, max_t, params,
num_vars_per_uav)
256     fprintf('\n--- 最佳协同策略 ---\n');
257     fprintf('最大总有效时长 (合并后): %.6f s\n', max_t);
258
259     fprintf('\n--- 各烟幕弹独立时长分析 ---\n');
260     all_final_intervals = [];
261     for i = 1:params.num_uavs
262         start_idx = (i-1)*num_vars_per_uav + 1;
263         v_i = best_vars(start_idx);
264         theta_i = best_vars(start_idx + 1);
265         td_i = best_vars(start_idx + 2);

```

```

266         tf_i = best_vars(start_idx + 3);
267
268         uav_vel_i = [v_i * cos(theta_i), v_i * sin(theta_i),
0];
269         pos_drop_i = params.uav_pos0(i,:) + uav_vel_i * td_i;
270         pos_deton_i = pos_drop_i + uav_vel_i * tf_i + 0.5 *
[0, 0, -params.g] * tf_i^2;
271         t_deton_i = td_i + tf_i;
272
273         intervals_i = calc_camo_intervals(t_deton_i,
pos_deton_i, params);
274         individual_duration = 0;
275         if ~isempty(intervals_i)
276             individual_duration = sum(intervals_i(:,2) -
intervals_i(:,1));
277             all_final_intervals = [all_final_intervals;
intervals_i];
278         end
279         fprintf('    - FY%d 产生的烟幕独立有效时长: %.4f s\n', i
, individual_duration);
280     end
281     recalculated_total_t = merge_intervals(all_final_intervals
);
282     fprintf('    - 以上时长合并后的总时长 (验证): %.6f s\n',
recalculated_total_t);
283
284     output_data = cell(params.num_uavs, 10);
285     for i = 1:params.num_uavs
286         start_idx = (i-1)*num_vars_per_uav + 1;
287         v_i = best_vars(start_idx);
288         theta_i = best_vars(start_idx + 1);
289         td_i = best_vars(start_idx + 2);
290         tf_i = best_vars(start_idx + 3);
291
292         dir_deg = rad2deg(theta_i);

```

```

293         if dir_deg < 0, dir_deg = dir_deg + 360; end
294
295         uav_vel_i = [v_i * cos(theta_i), v_i * sin(theta_i),
0];
296         pos_drop_i = params.uav_pos0(i,:) + uav_vel_i * td_i;
297         pos_deton_i = pos_drop_i + uav_vel_i * tf_i + 0.5 *
[0, 0, -params.g] * tf_i^2;
298
299         output_data{i, 1} = sprintf('FY%d', i);
300         output_data{i, 2} = dir_deg;
301         output_data{i, 3} = v_i;
302         output_data{i, 4} = pos_drop_i(1);
303         output_data{i, 5} = pos_drop_i(2);
304         output_data{i, 6} = pos_drop_i(3);
305         output_data{i, 7} = pos_deton_i(1);
306         output_data{i, 8} = pos_deton_i(2);
307         output_data{i, 9} = pos_deton_i(3);
308
309         fprintf('\n--- 无人机 %s 策略 ---\n', output_data{i,
1});
310         fprintf('    - 速度: %.4f m/s,    方向: %.4f°\n', v_i,
dir_deg);
311         fprintf('    - 投放时间 (td): %.4f s,    引信时间 (tf):
%.4f s\n', td_i, tf_i);
312         fprintf('    - 投放点坐标: [%.2f, %.2f, %.2f]\n',
pos_drop_i);
313         fprintf('    - 起爆点坐标: [%.2f, %.2f, %.2f]\n',
pos_deton_i);
314     end
315
316     output_data{1, 10} = max_t;
317     for i = 2:params.num_uavs, output_data{i, 10} = NaN; end
318
319     col_names = {
320         '无人机编号', '无人机运动方向', '无人机运动速度_ms',

```

```

...
321     '投放点_x', '投放点_y', '投放点_z', ...
322     '起爆点_x', '起爆点_y', '起爆点_z', ...
323     '总有效干扰时长_s'
324 };
325 result_table = cell2table(output_data, 'VariableNames',
col_names);
326 filename = 'result_qga_ultimate.xlsx';
327 writetable(result_table, filename, 'WriteVariableNames',
true);
328 fprintf('\n结果已保存至 %s 文件中。 \n', filename);
329 end
330
331 function intervals = calc_camo_intervals(t_deton, pos_deton,
params)
332     intervals = [];
333     if pos_deton(3) < 0, return; end
334     t_vec = linspace(t_deton, t_deton + params.smoke_dur, 20);
335     is_camo = false(size(t_vec));
336     for i = 1:length(t_vec)
337         is_camo(i) = is_fully_camo(t_vec(i), t_deton,
pos_deton, params);
338     end
339     if ~any(is_camo), return; end
340     diff_vec = diff([false, is_camo, false]);
341     start_idx = find(diff_vec == 1);
342     end_idx = find(diff_vec == -1);
343     for i = 1:length(start_idx)
344         t_low = t_vec(max(1, start_idx(i) - 1));
345         t_high = t_vec(min(length(t_vec), end_idx(i)-1));
346         t_start_prec = find_prec_bound(t_low, t_high, true,
t_deton, pos_deton, params);
347         t_end_prec = find_prec_bound(t_low, t_high, false,
t_deton, pos_deton, params);
348         if t_end_prec > t_start_prec

```

```

349         intervals = [intervals; t_start_prec, t_end_prec];
350     end
351 end
352 end
353
354 function t_prec = find_prec_bound(t_low, t_high, find_start,
    t_det, pos_det, params)
355     for iter = 1:15
356         t_mid = (t_low + t_high) / 2;
357         is_mid = is_fully_camo(t_mid, t_det, pos_det, params);
358         if find_start
359             if is_mid, t_high = t_mid; else, t_low = t_mid;
360         end
361         else
362             if is_mid, t_low = t_mid; else, t_high = t_mid;
363         end
364     end
365     if find_start, t_prec = t_high; else, t_prec = t_low; end
366 end
367
368 function total_dur = merge_intervals(intervals)
369     if isempty(intervals), total_dur = 0; return; end
370     sorted = sortrows(intervals, 1);
371     merged = sorted(1,:);
372     for i = 2:size(sorted, 1)
373         if sorted(i,1) <= merged(end,2)
374             merged(end,2) = max(merged(end,2), sorted(i,2));
375         else
376             merged = [merged; sorted(i,:)];
377         end
378     end
379     total_dur = sum(merged(:,2) - merged(:,1));
380 end

```

```

381 function is_camo = is_fully_camo(t, t_det, pos_det, params)
382     missile_pos = params.M1_pos0 + params.M1_vel_vec * t;
383     cloud_pos = pos_det - [0, 0, params.smoke_sink_v * (t -
    t_det)];
384     for i = 1:size(params.target_pts, 1)
385         if pt_seg_dist(cloud_pos, missile_pos, params.
    target_pts(i, :)) > params.smoke_r
386             is_camo = false;
387             return;
388         end
389     end
390     is_camo = true;
391 end
392
393 function dist = pt_seg_dist(p0, p1, p2)
394     v = p2 - p1;
395     w = p0 - p1;
396     c1 = dot(w, v);
397     if c1 <= 0, dist = norm(p0 - p1); return; end
398     c2 = dot(v, v);
399     if c2 <= c1, dist = norm(p0 - p2); return; end
400     b = c1 / c2;
401     pb = p1 + b * v;
402     dist = norm(p0 - pb);
403 end
404
405 function pts = generate_cyl_pts(N, r, b_center, h)
406     theta_out = linspace(0, 2*pi, N+1)';
407     theta_out = theta_out(1:end-1);
408     x = r*cos(theta_out);
409     y = r*sin(theta_out);
410     z = zeros(length(theta_out), 1);
411     bottom_pts = b_center + [x, y, z];
412     top_pts = b_center + [x, y, z + h];
413     pts = [bottom_pts; top_pts];

```

414 end

solve5.m

```
1 function solve()
2 clear; clc; close all;
3 fprintf('=== 第五问最终求解程序（搜索-计分分离版） ===\n\n');
4
5 try
6     delete(gcp('nocreate'));
7     parpool('local', 8);
8     fprintf('已启动8核并行计算池。 \n');
9 catch
10     fprintf('警告： 无法创建新的并行计算池。 \n');
11 end
12
13 params = initialize_params();
14
15
16 fprintf('\n--- [阶段一] 独立优化各无人机（采用原始搜索逻辑）
17     ---\n');
18 solutions = cell(params.NUM_UAVS, 1);
19 parfor uav_id = 1:params.NUM_UAVS
20     fprintf('--> 开始优化无人机 FY%d...\n', uav_id);
21     solutions{uav_id} = optimize_uav(uav_id, params);
22 end
23
24 fprintf('\n\n--- [阶段二] 全局汇总与最终计分（采用修正后评估逻辑） ---\n');
25 [final_plans, final_total_time, time_breakdown] =
26     combine_and_evaluate_new_logic(solutions, params);
27
28 display_plan_table(final_plans, params);
29
```

```

30 fprintf('\n--- 最终结果分析 ---\n');
31 fprintf('由原始搜索逻辑计算出的总时长（区间取并集）: %.4f s\n',
        calculate_old_logic_time(final_plans, params));
32 fprintf(2, '\n--- 基于正确评分标准的真实效率评估 ---\n');
33 fprintf('对 M1 的总有效遮蔽时长（合并后）: %.4f s\n',
        time_breakdown(1));
34 fprintf('对 M2 的总有效遮蔽时长（合并后）: %.4f s\n',
        time_breakdown(2));
35 fprintf('对 M3 的总有效遮蔽时长（合并后）: %.4f s\n',
        time_breakdown(3));
36 fprintf(2, '真实协同总时长 (T_M1+T_M2+T_M3): %.4f s\n',
        final_total_time);
37
38 generate_excel_report(final_plans, params, final_total_time);
39 plot_3d_scene(final_plans, params);
40
41 delete(gcf('nocreate'));
42 fprintf('并行计算池已关闭。 \n');
43 fprintf('\n执行完毕 \n');
44 end
45
46
47
48 function [final_plans, final_total_time, time_breakdown] =
        combine_and_evaluate_new_logic(solutions, params)
49     all_plans = [];
50     for i = 1:length(solutions)
51         sol = solutions{i};
52         if isempty(sol) || ~isfield(sol, 'grenade_plans') ||
            isempty(sol.grenade_plans)
53             continue;
54         end
55         all_plans = [all_plans; sol.grenade_plans];
56     end
57     final_plans = all_plans;

```

```

58     if isempty(final_plans)
59         final_total_time = 0;
60         time_breakdown = [0, 0, 0];
61         return;
62     end
63
64     intervals_by_missile = cell(params.NUM_MISSILES, 1);
65
66     for i = 1:length(final_plans)
67         plan = final_plans(i);
68         t_det = plan.td + plan.tf;
69
70
71         for m_id = 1:params.NUM_MISSILES
72             missile = params.missile(m_id);
73             if t_det > missile.flight_time, continue; end
74
75             intervals = calculate_camouflage_intervals(t_det,
plan.p_det, missile, params);
76             intervals_by_missile{m_id} = [intervals_by_missile
{m_id}; intervals];
77         end
78     end
79
80
81     final_total_time = 0;
82     time_breakdown = zeros(1, params.NUM_MISSILES);
83     for k = 1:params.NUM_MISSILES
84         duration_k = calculate_duration_from_intervals(
intervals_by_missile{k});
85         time_breakdown(k) = duration_k;
86         final_total_time = final_total_time + duration_k;
87     end
88 end
89

```

```

90 function old_total_time = calculate_old_logic_time(final_plans
    , params)
91     all_intervals_in_one_basket = [];
92     for i = 1:length(final_plans)
93         plan = final_plans(i);
94         t_det = plan.td + plan.tf;
95         for missile_id = 1:params.NUM_MISSILES
96             missile = params.missile(missile_id);
97             if t_det > missile.flight_time, continue; end
98             intervals = calculate_camouflage_intervals(t_det,
plan.p_det, missile, params);
99             all_intervals_in_one_basket = [
all_intervals_in_one_basket; intervals];
100         end
101     end
102     old_total_time = calculate_duration_from_intervals(
all_intervals_in_one_basket);
103 end
104
105 function final_plan = optimize_uav(uav_id, global_params)
106     params = global_params;
107     params.uav_pos0 = global_params.UAV_INIT_POS(uav_id, :);
108     params.uav_id = uav_id;
109     vec_to_target = params.M2_center_bottom(1:2) - params.
uav_pos0(1:2);
110     params.theta_base = atan2(vec_to_target(2), vec_to_target
(1));
111
112     pop_size = 200; max_gen = 2000; elite_count = 10;
113     mutation_rate_base = 0.5; crossover_rate = 0.8;
114     stagnation_limit = 80;
115
116     uav_alt = params.uav_pos0(3); t_flight_max = sqrt(2*
uav_alt/params.g);
117     t_mission_max = 62; delta_t_max = 25;

```

```

118     lb = [70, -pi, 0.1, 0.1, 1.0, 0.1, 1.0, 0.1];
119     ub = [140, pi, t_mission_max, t_flight_max, delta_t_max,
t_flight_max, delta_t_max, t_flight_max];
120
121     population = zeros(pop_size, length(lb));
122     seed = [140, 0, 0.1, 3.78, 3.26, 5.30, 4.26, 5.80];
123     population(1, :) = repair_solution(seed, lb, ub);
124
125     num_seeded = round(pop_size / 3);
126     for i = 2:num_seeded, population(i, :) = mutate(population
(1, :), lb, ub, 0.5); end
127     num_mutated_seed = round(pop_size / 3);
128     for i = (num_seeded + 1):(num_seeded + num_mutated_seed),
population(i, :) = mutate(population(1, :), lb, ub, 1.2);
end
129     for i = (num_seeded + num_mutated_seed + 1):pop_size,
population(i, :) = lb + (ub - lb) .* rand(1, length(lb));
end
130
131     history_best_fitness = zeros(max_gen, 1);
132     stagnation_count = 0; last_best_fitness = -1;
133     last_reset_elite_pos = [];
134
135     for gen = 1:max_gen
136         fitness = zeros(pop_size, 1);
137         parfor i = 1:pop_size
138             fitness(i) = objective_old_logic(population(i, :),
params);
139         end
140
141         [sorted_fitness, sorted_idx] = sort(fitness, 'descend'
);
142         history_best_fitness(gen) = sorted_fitness(1);
143
144         if abs(history_best_fitness(gen) - last_best_fitness)

```

```

< 1e-4, stagnation_count = stagnation_count + 1; else,
stagnation_count = 0; end
145     last_best_fitness = history_best_fitness(gen);
146
147     if stagnation_count >= stagnation_limit
148         fprintf('  FY%d - 停滞，触发重置机制!\n', uav_id);
149         sorted_pop = population(sorted_idx, :); elites =
sorted_pop(1:elite_count, :); current_best_pos = elites(1,
:);
150
151         num_random = round((pop_size - elite_count) * 0.3)
;
152         num_repel = round((pop_size - elite_count) * 0.3);
153         num_shock = pop_size - elite_count - num_random -
num_repel;
154
155         random_immigrants = lb + (ub - lb) .* rand(
num_random, length(lb));
156
157         repelled_solutions = zeros(num_repel, length(lb));
158         if ~isempty(last_reset_elite_pos) && norm(
current_best_pos - last_reset_elite_pos) < 1e-2
159             fprintf('    从重复停滞点进行排斥.\n');
160             for k = 1:num_repel, r_dir = rand(1, length(lb
)) - 0.5; r_dir = r_dir / norm(r_dir); repelled_solutions(k,
:) = current_best_pos + r_dir .* (ub-lb) * 0.7; end
161         else
162             repelled_solutions = lb + (ub - lb) .* rand(
num_repel, length(lb));
163         end
164
165         shock_mutants = zeros(num_shock, length(lb));
166         for k = 1:num_shock
167             seed_ind = elites(randi(elite_count), :);
dims_to_shock = [2, 3, 5, 7]; num_dims = randi([2, 3]);

```

```

dims_chg = randsample(dims_to_shock, num_dims);
168         for d_idx = 1:length(dims_chg), dim = dims_chg
(d_idx); seed_ind(dim) = lb(dim) + (ub(dim) - lb(dim)) *
rand(); end
169         shock_mutants(k, :) = seed_ind;
170     end
171
172     population = [elites; random_immigrants;
repelled_solutions; shock_mutants];
173     population = repair_population(population, lb, ub)
;
174     last_reset_elite_pos = current_best_pos;
175     stagnation_count = 0; last_best_fitness = -1;
176     continue;
177 end
178
179     sorted_population = population(sorted_idx, :);
180     next_population = zeros(size(population));
181     next_population(1:elite_count, :) = sorted_population
(1:elite_count, :);
182
183     for i = (elite_count + 1):pop_size
184         p1_idx = tournament_selection(pop_size); p2_idx =
tournament_selection(pop_size);
185         parent1 = sorted_population(p1_idx, :); parent2 =
sorted_population(p2_idx, :);
186
187         if rand < crossover_rate, child = crossover_blend(
parent1, parent2); else, child = parent1; end
188
189         dyn_mut_rate = mutation_rate_base + 0.3 * (
stagnation_count / stagnation_limit);
190         if rand < min(dyn_mut_rate, 0.85)
191             mut_scale = 1 - (gen/max_gen)^2 + 0.5 * (
stagnation_count / stagnation_limit);

```

```

192         child = mutate(child, lb, ub, min(mut_scale,
1.2));
193     end
194     next_population(i, :) = repair_solution(child, lb,
ub);
195 end
196 population = next_population;
197
198     if mod(gen, 50) == 0 || gen == 1, fprintf('  FY%d-代数
%d/%d, 最优时长: %.4fs\n', uav_id, gen, max_gen,
history_best_fitness(gen)); end
199 end
200
201     best_chromosome = population(sorted_idx(1), :);
202     final_plan = decode_chromosome(best_chromosome, params);
203     final_plan.fitness = history_best_fitness(end);
204     fprintf('<-- 完成FY%d优化, 个体最优时长: %.4fs\n', uav_id,
final_plan.fitness);
205 end
206
207 function total_time = objective_old_logic(x, params)
208     v = x(1); delta_theta = x(2);
209     uav_vel = [v*cos(params.theta_base+delta_theta), v*sin(
params.theta_base+delta_theta), 0];
210     td1 = x(3); tf1 = x(4); dt12 = x(5); tf2 = x(6); dt23 = x
(7); tf3 = x(8);
211     td2 = td1 + dt12; td3 = td2 + dt23;
212     drops = [td1, tf1; td2, tf2; td3, tf3];
213     all_intervals = [];
214     for smoke_id = 1:3
215         td = drops(smoke_id, 1); tf = drops(smoke_id, 2);
216         p_drop = params.uav_pos0 + uav_vel * td;
217         p_det = p_drop + uav_vel * tf + 0.5 * [0, 0, -params.g
] * tf^2;
218         t_det = td + tf;

```

```

219         for missile_id = 1:params.NUM_MISSILES
220             missile = params.missile(missile_id);
221             if t_det > missile.flight_time, continue; end
222             intervals = calculate_camouflage_intervals(t_det,
p_det, missile, params);
223             all_intervals = [all_intervals; intervals];
224         end
225     end
226     total_time = calculate_duration_from_intervals(
all_intervals);
227 end
228
229 function plan_struct = decode_chromosome(x, params)
230     plan_struct.uav_id = params.uav_id;
231     plan_struct.v = x(1);
232     delta_theta = x(2);
233     plan_struct.theta_rad = params.theta_base + delta_theta;
234     plan_struct.theta_deg = rad2deg(plan_struct.theta_rad);
235     if plan_struct.theta_deg < 0, plan_struct.theta_deg =
plan_struct.theta_deg + 360; end
236
237     td1 = x(3); tf1 = x(4); dt12 = x(5); tf2 = x(6); dt23 = x
(7); tf3 = x(8);
238     td2 = td1 + dt12; td3 = td2 + dt23;
239     drops = [td1, tf1; td2, tf2; td3, tf3];
240
241     uav_vel = [plan_struct.v*cos(plan_struct.theta_rad),
plan_struct.v*sin(plan_struct.theta_rad), 0];
242     grenade_plans_list = [];
243
244     for i = 1:3
245         td = drops(i, 1); tf = drops(i, 2);
246         p_drop = params.uav_pos0 + uav_vel * td;
247         p_det = p_drop + uav_vel * tf + 0.5 * [0, 0, -params.g
] * tf^2;

```

```

248         t_det = td + tf;
249
250         temp_intervals = [];
251         for m_id = 1:params.NUM_MISSILES
252             if t_det > params.missile(m_id).flight_time,
continue; end
253             intervals = calculate_camouflage_intervals(t_det,
p_det, params.missile(m_id), params);
254             temp_intervals = [temp_intervals; intervals];
255         end
256
257         time_contrib = calculate_duration_from_intervals(
temp_intervals);
258         if time_contrib > 1e-4
259             plan.uav_id = params.uav_id;
260             plan.missile_id = find_best_nominal_target(t_det,
p_det, params);
261             plan.td = td; plan.tf = tf;
262             plan.v = plan_struct.v; plan.theta_deg =
plan_struct.theta_deg;
263             plan.p_drop = p_drop; plan.p_det = p_det;
264             grenade_plans_list = [grenade_plans_list; plan];
265         end
266     end
267     plan_struct.grenade_plans = grenade_plans_list;
268 end
269
270 function best_m_id = find_best_nominal_target(t_det, p_det,
params)
271     max_dur = -1; best_m_id = 1;
272     for m_id = 1:params.NUM_MISSILES
273         intervals = calculate_camouflage_intervals(t_det,
p_det, params.missile(m_id), params);
274         dur = calculate_duration_from_intervals(intervals);
275         if dur > max_dur, max_dur = dur; best_m_id = m_id; end

```

```

276     end
277 end
278
279
280 function pop = repair_population(pop, lb, ub), for i = 1:size(
    pop, 1), pop(i, :) = repair_solution(pop(i, :), lb, ub); end
    , end
281 function x = repair_solution(x, lb, ub), x = max(lb, min(ub,
    reshape(x, 1, []))); if x(5) < 1.0, x(5) = 1.0; end; if x(7)
    < 1.0, x(7) = 1.0; end, end
282 function child = crossover_blend(p1, p2), alpha = 0.5; diff =
    abs(p1 - p2); lb = min(p1, p2) - alpha * diff; ub = max(p1,
    p2) + alpha * diff; child = lb + (ub - lb) .* rand(size(p1))
    ; end
283 function x_mut = mutate(x, lb, ub, scale), x_mut = x; dim =
    randi(length(x)); step_range = (ub(dim) - lb(dim)) * 0.15 *
    scale; step = step_range * (2 * rand() - 1); x_mut(dim) = x(
    dim) + step; end
284 function idx = tournament_selection(pop_size, tour_size), if
    nargin < 2, tour_size = 3; end; cands = randi(pop_size, 1,
    tour_size); idx = min(cands); end
285 function params = initialize_params(), params.g = 9.8; params.
    smoke_radius = 10; params.smoke_sink_vel = 3; params.
    smoke_duration = 20; params.M2_center_bottom = [0, 200, 0];
    missile_pos0 = [2000, 0, 2000; 19000, 600, 2100; 18000,
    -600, 1900]; params.NUM_MISSILES = 3; params.missile =
    struct(); for i = 1:params.NUM_MISSILES, params.missile(i).
    pos0 = missile_pos0(i, :); params.missile(i).vel_vec = (
    params.missile(i).pos0 * -1) / norm(params.missile(i).pos0)
    * 300; params.missile(i).flight_time = norm(params.missile(i)
    ).pos0) / 300; end; params.UAV_INIT_POS = [17800, 0, 1800;
    12000, 1400, 1400; 6000, -3000, 700; 11000, 2000, 1800;
    13000, -2000, 1300]; params.NUM_UAVS = 5; params.
    MAX_GRENADES_PER_UAV = 3; N_target_points = 30; params.
    target_points_for_check = generate_cylinder_points(

```

```

N_target_points, 7, params.M2_center_bottom, 10); end
286 function intervals = calculate_camouflage_intervals(t_det,
    pos_det, missile, params), intervals = []; if pos_det(3) <
    0, return; end; t_sim_start = t_det; t_sim_end = t_det +
    params.smoke_duration; N_coarse = 20; time_vector_coarse =
    linspace(t_sim_start, t_sim_end, N_coarse);
    is_camouflaged_coarse = false(1, N_coarse); for i = 1:
    N_coarse, is_camouflaged_coarse(i) =
    is_fully_camouflaged_at_t(time_vector_coarse(i), t_det,
    pos_det, missile, params); end; if ~any(
    is_camouflaged_coarse), return; end; diff_vec = diff([false,
    is_camouflaged_coarse, false]); start_indices = find(
    diff_vec == 1); end_indices = find(diff_vec == -1);
    N_bisection = 15; for i = 1:length(start_indices), idx_low =
    max(1, start_indices(i) - 1); idx_high = min(N_coarse,
    end_indices(i) - 1); t_low = time_vector_coarse(idx_low);
    t_high = time_vector_coarse(idx_high); t_start_precise =
    find_precise_boundary(t_low, t_high, true, N_bisection,
    t_det, pos_det, missile, params); t_end_precise =
    find_precise_boundary(t_low, t_high, false, N_bisection,
    t_det, pos_det, missile, params); if t_end_precise >
    t_start_precise, intervals = [intervals; t_start_precise,
    t_end_precise]; end; end, end
287 function t_precise = find_precise_boundary(t_low, t_high,
    find_start, N_iter, t_det, pos_det, missile, params), for i
    = 1:N_iter, t_mid = (t_low + t_high) / 2; is_mid_shaded =
    is_fully_camouflaged_at_t(t_mid, t_det, pos_det, missile,
    params); if find_start, if is_mid_shaded, t_high = t_mid;
    else, t_low = t_mid; end; else, if is_mid_shaded, t_low =
    t_mid; else, t_high = t_mid; end; end; end; if find_start,
    t_precise = t_high; else, t_precise = t_low; end, end
288 function is_camouflaged = is_fully_camouflaged_at_t(t,
    t_detonation, pos_detonation, missile, params),
    missile_pos_t = missile.pos0 + missile.vel_vec * t;
    cloud_pos_t = pos_detonation - [0, 0, params.smoke_sink_vel

```

```

* (t - t_detonation)]; for i = 1:size(params.
target_points_for_check, 1), p_target = params.
target_points_for_check(i, :); dist_to_sightline =
point_to_segment_dist(cloud_pos_t, missile_pos_t, p_target);
    if dist_to_sightline > params.smoke_radius, is_camouflaged
= false; return; end; end; is_camouflaged = true; end
289 function merged = get_merged_intervals(intervals), if isempty(
intervals), merged = []; return; end; sorted_intervals =
sortrows(intervals, 1); merged = sorted_intervals(1, :); for
i = 2:size(sorted_intervals, 1), current_interval =
sorted_intervals(i, :); last_merged_end = merged(end, 2); if
current_interval(1) <= last_merged_end, merged(end, 2) =
max(last_merged_end, current_interval(2)); else, merged = [
merged; current_interval]; end; end, end
290 function total_duration = calculate_duration_from_intervals(
intervals), merged = get_merged_intervals(intervals); if
isempty(merged), total_duration = 0; return; end;
total_duration = sum(merged(:, 2) - merged(:, 1)); end
291 function dist = point_to_segment_dist(p0, p1, p2), v = p2 - p1
; w = p0 - p1; c1 = dot(w, v); if c1 <= 0, dist = norm(p0 -
p1); return; end; c2 = dot(v, v); if c2 <= c1, dist = norm(
p0 - p2); return; end; b = c1 / c2; pb = p1 + b * v; dist =
norm(p0 - pb); end
292 function points = generate_cylinder_points(N, radius, b_center
, height), theta = linspace(0, 2 * pi, N + 1)'; theta =
theta(1:end - 1); x = radius * cos(theta); y = radius * sin(
theta); z = zeros(length(theta), 1); bottom_points =
b_center + [x, y, z]; top_pts = b_center + [x, y, z + height
]; points = [bottom_points; top_pts]; end
293
294
295 function display_plan_table(final_plans, ~)
296     fprintf('\n--- 最终协同策略详情 ---\n');
297     fprintf('%s\n', repmat('-', 100));
298     fprintf('%-12s | %-12s | %-8s | %-8s | %-8s | %-8s | %-8s\

```

```

n', '发射平台', '名义目标', '速度', '方向', '投放时间', '引
信', '起爆时间');
299     fprintf('%s\n', repmat('-', 100));
300     if isempty(final_plans), fprintf('未找到任何有效的投放计
划。 \n'); fprintf('%s\n', repmat('-', 100)); return; end
301
302     [~, sort_idx] = sort([final_plans.uav_id] * 1000 + [
final_plans.td]);
303     final_plans = final_plans(sort_idx);
304
305     for i = 1:length(final_plans)
306         plan = final_plans(i);
307         t_det = plan.td + plan.tf;
308         fprintf('FY%d-弹%d | M%-12d | %-8.2f | %-8.2f | %-8.2f
| %-8.2f | %-8.2f\n', ...
309             plan.uav_id, i, plan.missile_id, plan.v, plan.
theta_deg, plan.td, plan.tf, t_det);
310     end
311     fprintf('%s\n', repmat('-', 100));
312 end
313
314 function generate_excel_report(final_plans, params, total_time
)
315     if isempty(final_plans), fprintf('没有有效的投放计划，无法
生成Excel报告。 \n'); return; end
316
317     col_headers = {'无人机编号', '总投弹序号', '名义目标', '速
度(m/s)', '方向(deg)', '投放时间(s)', '引信时间(s)', '投放点
X(m)', '投放点Y(m)', '投放点Z(m)', '起爆点X(m)', '起爆点Y(m)
', '起爆点Z(m)', '对M1贡献(s)', '对M2贡献(s)', '对M3贡献(s)'
};
318     output_data = cell(length(final_plans), length(col_headers
));
319
320     [~, sort_idx] = sort([final_plans.uav_id] * 1000 + [

```

```

final_plans.td]);
321     final_plans = final_plans(sort_idx);
322
323     for i = 1:length(final_plans)
324         plan = final_plans(i);
325         t_det = plan.td + plan.tf;
326
327         output_data{i, 1} = sprintf('FY%d', plan.uav_id);
328         output_data{i, 2} = i;
329         output_data{i, 3} = sprintf('M%d', plan.missile_id);
330         output_data{i, 4} = plan.v;
331         output_data{i, 5} = plan.theta_deg;
332         output_data{i, 6} = plan.td;
333         output_data{i, 7} = plan.tf;
334         output_data{i, 8} = plan.p_drop(1);
335         output_data{i, 9} = plan.p_drop(2);
336         output_data{i, 10} = plan.p_drop(3);
337         output_data{i, 11} = plan.p_det(1);
338         output_data{i, 12} = plan.p_det(2);
339         output_data{i, 13} = plan.p_det(3);
340
341         contribs = zeros(1, params.NUM_MISSILES);
342         for m_id = 1:params.NUM_MISSILES
343             missile = params.missile(m_id);
344             if t_det > missile.flight_time, continue; end
345             intervals = calculate_camouflage_intervals(t_det,
plan.p_det, missile, params);
346             contribs(m_id) = calculate_duration_from_intervals
(intervals);
347         end
348         output_data{i, 14} = contribs(1);
349         output_data{i, 15} = contribs(2);
350         output_data{i, 16} = contribs(3);
351     end
352

```

```

353     T = cell2table(output_data, 'VariableNames', col_headers);
354
355     intervals_by_missile = cell(params.NUM_MISSILES, 1);
356     for i = 1:length(final_plans)
357         plan = final_plans(i);
358         t_det = plan.td + plan.tf;
359         for m_id = 1:params.NUM_MISSILES
360             missile = params.missile(m_id);
361             if t_det > missile.flight_time, continue; end
362             intervals = calculate_camouflage_intervals(t_det,
plan.p_det, missile, params);
363             intervals_by_missile{m_id} = [intervals_by_missile
{m_id}; intervals];
364         end
365     end
366
367     T_M1 = calculate_duration_from_intervals(
intervals_by_missile{1});
368     T_M2 = calculate_duration_from_intervals(
intervals_by_missile{2});
369     T_M3 = calculate_duration_from_intervals(
intervals_by_missile{3});
370
371     summary_header = {'协同效果汇总', ''};
372     summary_data = {'对M1总有效时长(s)', T_M1; '对M2总有效时长
(s)', T_M2; '对M3总有效时长(s)', T_M3; '最终协同总时长(s)',
total_time};
373
374     filename = '第五问最终策略报告.xlsx';
375     writetable(T, filename, 'Sheet', '详细策略', '
WriteVariableNames', true);
376
377     summary_start_row = size(T, 1) + 3;
378     writecell(summary_header, filename, 'Sheet', '详细策略', '
Range', sprintf('A%d', summary_start_row));

```

```

379     writecell(summary_data, filename, 'Sheet', '详细策略', '
Range', sprintf('A%d', summary_start_row + 1));
380
381     fprintf('详细报告已保存至 %s\n', filename);
382 end
383
384 function plot_3d_scene(final_plans, config)
385     if isempty(final_plans), return; end
386
387     figure('Name', '最终协同策略三维视图', 'Position', [50,
50, 1200, 900]);
388     hold on; grid on; axis equal; view(60, 25);
389     xlabel('X (m)'); ylabel('Y (m)'); zlabel('Z (m)');
390     title('最终协同策略三维视图');
391
392     [Xc, Yc, Zc] = cylinder(7, 30); Zc = Zc * 10;
393     surf(Xc + config.M2_center_bottom(1), Yc + config.
M2_center_bottom(2), Zc, 'FaceColor', 'g', 'EdgeColor', '
none', 'FaceAlpha', 0.7, 'DisplayName', '真目标 (M2)');
394     plot3(0, 0, 0, 'pk', 'MarkerSize', 15, 'MarkerFaceColor',
'k', 'DisplayName', '假目标');
395
396     missile_colors = {[1, 0, 0], [1, 0.5, 0], [0.8, 0, 0.2]};
397     for i = 1:config.NUM_MISSILES
398         p_start = config.missile(i).pos0;
399         plot3([p_start(1), 0], [p_start(2), 0], [p_start(3),
0], '--', 'Color', missile_colors{i}, 'LineWidth', 2, '
DisplayName', sprintf('M%d 弹道', i));
400     end
401
402     uav_colors = {'b', 'c', 'm', [0.5, 0.5, 1], [0, 0.5,
0.5]};
403     all_uav_ids = unique([final_plans.uav_id]);
404     for i = 1:length(all_uav_ids)
405         uav_id = all_uav_ids(i);

```

```

406     uav_p0 = config.UAV_INIT_POS(uav_id, :);
407     tasks = final_plans([final_plans.uav_id] == uav_id);
408     if isempty(tasks), continue; end
409
410     max_td = max([tasks.td]);
411     uav_vel = [tasks(1).v * cosd(tasks(1).theta_deg),
412 tasks(1).v * sind(tasks(1).theta_deg), 0];
413     p_end = uav_p0 + uav_vel * max_td;
414     plot3([uav_p0(1), p_end(1)], [uav_p0(2), p_end(2)], [
415 uav_p0(3), p_end(3)], '-', 'Color', uav_colors{uav_id}, '
416 LineWidth', 1.5, 'DisplayName', sprintf('FY%d 轨迹', uav_id)
417 );
418 end
419
420 [Xs, Ys, Zs] = sphere(20);
421 for i = 1:length(final_plans)
422     plan = final_plans(i);
423     p_drop = plan.p_drop;
424     plot3([p_drop(1), plan.p_det(1)], [p_drop(2), plan.
425 p_det(2)], [p_drop(3), plan.p_det(3)], ':', 'Color',
426 uav_colors{plan.uav_id}, 'HandleVisibility', 'off');
427     surf(Xs * 10 + plan.p_det(1), Ys * 10 + plan.p_det(2),
428 Zs * 10 + plan.p_det(3), 'FaceColor', uav_colors{plan.
429 uav_id}, 'EdgeColor', 'none', 'FaceAlpha', 0.3, '
430 HandleVisibility', 'off');
431 end
432
433 legend('show', 'Location', 'bestoutside');
434 hold off;
435 end

```